Non-Negative Stiefel Approximating Flow: Orthogonalish Matrix Optimization for Interpretable Embeddings

Brian B. Avants, Nicholas J. Tustison, and James R. Stone
Department of Radiology and Medical Imaging
University of Virginia, Charlottesville, VA 22903

October 11, 2025

Abstract

NSA-Flow estimates a matrix that is non-negative and approximately orthogonal, with the degree of approximation set by one parameter. For a non-negative matrix, orthogonal columns have disjoint supports, so each component names a set of features and no feature is shared. Exact disjointness is a clustering of the features and discards the data's ability to say that two features belong together; the relaxation is what makes the interior of that constraint reachable. We keep non-negativity as a hard constraint and treat orthogonality as an objective weighted by $w \in [0, 1]$, so $w = 0$ returns the unconstrained fit and $w = 1$ returns the clustering. Both terms of the energy are dimensionless, so w needs no calibration. The solver uses bound-constrained L-BFGS-B or spectral projected gradient with a computable stationarity certificate and no factorisation in the loop, at $O(pk^2)$ per iteration, or $O(npk)$ when the basis is fitted to data with $p > n$. NSA-Flow can refine a supplied basis, fit one to data directly, or act as the proximal step inside another algorithm. On the Golub leukemia data it achieves 56.9% sparsity and 0.003 orthogonality defect while matching dense PCA on in-fold binary cross-validated accuracy (0.800 vs. 0.807, and 0.795 for soft-thresholding sparse PCA), while signed NSA-Flow raises 3-class leukemia balanced accuracy from 0.698 to 0.748. On ADNI cortical thickness it raises out-of-sample ΔR^2 for nine cognitive outcomes by 0.041 over PCA (+0.123 on CDRSB clinical progression) and trinary diagnosis AUC from 0.716 to 0.732 ($p = 1.24 \times 10^{-4}$). We also show that comparisons of this kind must use a model that is sensitive to the choice of axes: a linear model on projected scores is exactly invariant to reparametrisation of the basis and cannot distinguish two bases spanning the same subspace.

1 Introduction

Data matrices in neuroscience, genomics and text encode overlapping sources of variation: voxels carry distributed activity, genes are co-expressed across pathways, words co-occur across topics. A basis fitted to such data spreads each component across most features, which makes the components hard to name. Two constraints make them nameable. Non-negativity turns loadings into magnitudes. Orthogonality, imposed on a non-negative matrix, forces the columns onto disjoint feature supports. Taken exactly, the pair is very rigid: disjoint supports are a hard assignment of features to components, so the constrained problem is a clustering and the data can no longer say that two features belong together. This paper is about the interior of that constraint, and about reaching it with one parameter.

Classical dimensionality reduction techniques like principal component analysis (PCA) and its sparse variants seek low-rank approximations with interpretable bases (Zou, Hastie, and Tibshirani 2006), while sparse canonical correlation analysis (CCA) extends this to multi-view correlations in biological data (Witten, Tibshirani, and Hastie 2009). However, enforcing sparsity and decorrelation remains challenging: traditional methods may over-regularize or lack intuitive controls for partial constraints. Non-negative matrix factorization (NMF) (Daniel D. Lee and Seung 2001b) offers parts-based, additive representations aligned with

domain constraints, but suffers from rotational ambiguity, yielding entangled factors (Ding et al. 2006). Orthogonal variants improve sparsity and identifiability by aligning factors with disjoint structures, with applications in sparse PCA (Zou, Hastie, and Tibshirani 2006), sparse CCA (Witten, Tibshirani, and Hastie 2009), and interpretable neural networks (Henaff et al. 2011). Yet, strict orthogonality often sacrifices fidelity, especially in noisy or heterogeneous data and motivates the need for partial decorrelation models.

Soft orthogonalization methods address this by penalizing deviations from orthonormality. Examples include Disentangled Orthogonality Regularization (DOR), which separates Gram matrix components for convolutional kernels (Wu 2023); Group Orthogonalization Regularization (GOR), which applies intra-group penalties for vision tasks (Kurtz, Bar, and Giryes 2023); λ-Orthogonality Regularization, which introduces thresholded penalties for representation learning (Ricci et al. 2025); and simpler approaches like Spectral Restricted Isometry Property (SRIP) (Goeßmann 2020) and Frobenius norm penalties for neural stability (Guo 2019). However, these methods are typically embedded in neural training pipelines and do not enforce non-negativity, limiting their applicability to domains where measurement units should be preserved. Advanced ONMF variants, such as variational Bayesian approaches (Rahiche et al. 2022), unilateral factorization (Li, Zhang, and Zhang 2023), and deep autoencoder frameworks (Yang and Xu 2021), improve robustness but enforce strict orthogonality or require full decomposition, reducing flexibility for one-sided refinement.

In contrast to these soft regularization techniques, Riemannian optimization approaches have been explored primarily for enforcing strict orthogonality constraints on manifolds such as the Stiefel manifold, where the feasible set is equipped with a differential structure for gradient-based updates. For instance, Nonlinear Riemannian Conjugate Gradient (NRCG) methods optimize orthogonal NMF by projecting gradients onto the tangent space and using retractions like QR decomposition to maintain exact orthonormality, while handling non-negativity through coordinate descent on the complementary factor (Zhang et al. 2016). This ensures convergence to critical points with near-perfect orthogonality but incurs higher computational costs compared to soft penalties, and it typically requires full enforcement rather than flexible deviations. Hybrid methods like Feedback Gradient Descent (FGD) attempt to bridge this gap by approximating manifold dynamics in Euclidean space with feedback terms to achieve stable near-orthogonality efficiently, outperforming traditional Riemannian methods in DNN training speed while rivaling soft constraints in overhead (Bu and Chang 2022). However, adapting such Riemannian-inspired techniques to incorporate soft orthogonality penalties alongside non-negativity remains a less explored avenue.

To address these gaps we propose **Non-negative Stiefel Approximating Flow (NSA-Flow)**, which approximates a target X_0 with a non-negative matrix $Y \in \mathbb{R}^{p\times k}_{\geq 0}$ whose columns are approximately orthogonal. One weight $w \in [0, 1]$ sets the balance between fidelity and orthogonality, and because both energy terms are dimensionless it needs no calibration. Non-negativity is a hard constraint applied by exact projection; orthogonality is an objective, so nothing is retracted and no factorisation appears in the iteration. For a non-negative matrix, orthogonal columns have disjoint supports, so w moves the solution between the unconstrained fit and a hard assignment of features to components. NSA-Flow can refine a basis that is already trusted, fit one to data directly, or serve as the proximal step inside another algorithm; the Golub result below is of the third kind.

Our contributions are:

1. A general framework for constrained matrix approximation, parameterized by (w) to intuitively control sparsity and orthogonality for interpretable ML.
2. Rigorous empirical validation that demonstrates good convergence properties and reliable benchmark performance compared to baselines.
3. Broad applications, including enhanced disease classification on the Golub leukemia dataset, non-negative sparse PCA for biological integration and interpretable brain network discovery—showcasing NSA-Flow's versatility across ML domains.
4. An open-source implementation in `pytorch` facilitating easy integration into existing workflows for researchers and practitioners. NSA-flow is also `pip` installable via the `nsa-flow` package and wrapped in `R` via `ANTsR`.

The paper is organized as follows: Section 2 derives the formulation and algorithm; Section 3 details the experimental results; Section 4 discusses limitations and future work; Section 5 gives an overview of software resources.

2 Methods

We consider the problem of finding a matrix $Y \in \mathbb{R}^{p\times k}$ that optimally approximates a target matrix $X_0 \in \mathbb{R}^{p\times k}$ while satisfying column orthogonality and, optionally, non-negativity. This general formulation is central to a wide range of problems in machine learning and signal processing, including orthogonal dictionary learning, Independent Component Analysis (ICA), and the orthogonal Procrustes problem (Daniel D. Lee and Seung 2001a; Hyvärinen and Oja 2000; Schönemann 1966). The NSA-Flow optimization problem is defined by the minimization of a composite energy function $E(Y)$:

$$\min_{Y\in\mathbb{R}^{p\times k}, Y\geq 0} E(Y) = (1-w)\, L_{fid}(Y, X_0) + w\, L_{orth}(Y),$$

where $w \in [0,1]$ balances the fidelity loss L_{fid} against the orthogonality loss L_{orth}. Both losses are dimensionless, so no re-weighting, warm-up calibration or data-dependent scaling constant is needed and w is a convex weight. The fidelity term is the relative squared Frobenius distance

$$L_{fid}(Y, X_0) = \frac{\|Y - X_0\|_F^2}{\|X_0\|_F^2},$$

which is 0 at $Y = X_0$ and exactly 1 at $Y = 0$.

For the orthogonality loss, let $G = Y^\top Y / \operatorname{tr}(Y^\top Y)$ be the trace-normalised Gram matrix. We use one of two penalties:

$$D(Y) = \left\| G - \tfrac{1}{k} I_k \right\|_F^2, \qquad C(Y) = \frac{1}{k(k-1)} \left\| \widehat{G} - I_k \right\|_F^2, \qquad \widehat{G}_{ij} = \langle \hat{y}_i, \hat{y}_j \rangle, \quad \hat{y}_i = \frac{y_i}{\max(\|y_i\|, \varepsilon)}.$$

The identity in C is not cosmetic and dropping it breaks the functional, so we write C out. For any column with $\|y_i\| \geq \varepsilon$ we have $\widehat{G}_{ii} = 1$ and $\widehat{G}_{ij} = \cos(y_i, y_j)$, giving

$$C(Y) = \frac{1}{k(k-1)} \Bigg[\underbrace{\sum_{i\neq j} \cos^2(y_i, y_j)}_{\text{pairwise angles}} + \underbrace{\sum_i \left(1 - \min\left(1, \|y_i\|^2/\varepsilon^2\right)\right)^2}_{\text{floored columns}} \Bigg],$$

where $\cos(y_i, y_j) = \langle y_i, y_j \rangle / (\|y_i\|\, \|y_j\|)$. The second sum is *exactly* zero on any Y whose columns all have norm at least ε, which is why the pairwise sum alone is the whole story in the generic case and why it is tempting to define C that way. It is not: each column whose norm has collapsed is charged $1/(k(k-1))$, so a Y with one live column and $k-1$ dead ones scores

$$C = \frac{k-1}{k(k-1)} = \frac{1}{k},$$

0.2 at $k = 5$ (row four of Table 1), whereas the pairwise sum alone scores it 0 — a dead column is orthogonal to everything. Defining C by the pairwise sum therefore reproduces the exact failure that rules out the off-diagonal defect (§2.1.3 and row four of Table 1), and any implementation written from that equation inherits it. In the code the two forms are `angle_defect(V, diagonal=True)` and `diagonal=False`. The variant without the identity, $\|\widehat{G} - \operatorname{diag}(\widehat{G})\|_F^2$, is used in exactly one place, the signed lifting (`nsa_flow.signed`), where an empty lobe is the correct answer for a one-signed component and the reconstruction term is what keeps the basis non-degenerate; everywhere in this paper C means the form with the identity.

D is scale invariant, lies in $[0, 1-1/k]$, and vanishes exactly on the scaled Stiefel manifold $\mathbb{R}_{>0} \cdot St(p,k)$; we report $\tilde{D} = D/(1-1/k)$ so that it lies in $[0,1]$ and equals 1 at rank one. It admits the spectral reading

$$D = k \operatorname{Var}(\lambda_i(G)) = \frac{1}{\operatorname{EffRank}(Y)} - \frac{1}{k},$$

so it is a participation-ratio deficit, and $D \geq 1/r - 1/k$ whenever $\operatorname{rank}(Y) = r < k$: rank collapse is penalised rather than rewarded.

D decomposes as

$$D = \underbrace{\sum_{i \neq j} G_{ij}^2}_{\text{decorrelation}} + \underbrace{\sum_i \left(G_{ii} - \tfrac{1}{k}\right)^2}_{\text{norm balance}},$$

so $D = 0$ requires mutual orthogonality and equal column norms, which is orthonormality. C requires orthogonality at any column norms. Use D when the components should carry equal weight and C when they should not: a disjoint non-negative basis whose columns differ in magnitude scores $D = 0.14$ and $C = 2 \times 10^{-33}$ (Table 1). When the basis is fitted to data the two behave the same, because the reconstruction term already equalises the column norms.

Table 1: The three candidate functionals on the structures that separate them. Zero means the functional recognises the structure as what it is looking for. Rows two and three are what this method exists to produce and only the last two columns score them at zero; row four is rank collapse and only the off-diagonal defect rewards it.

case	D (orthonormality)	off-diagonal only	C (orthogonality)
orthonormal	0.0000	0.0	0.0
orthogonal, unequal norms	0.6867	0.0	0.0
disjoint nonneg supports	0.1357	0.0	0.0
rank collapse (1 live col)	0.8000	0.0	0.2
all columns collinear	0.8000	0.8	1.0

The Euclidean gradient of the $\tilde{D}$ form is available in closed form,

$$\nabla_Y E(Y) = (1-w)\,\frac{2(Y - X_0)}{\|X_0\|_F^2} + \frac{w}{1-1/k} \cdot \frac{4}{t^2}\left[YS - \frac{\|S\|_F^2}{t} Y\right], \qquad S = Y^\top Y,\ t = \operatorname{tr} S,$$

requiring no factorisation. It satisfies the Euler identity $\langle \nabla D, Y \rangle = 0$, since D is homogeneous of degree zero, so the orthogonality term cannot alter $\|Y\|_F$; and $\|\nabla D\|_F \leq 8/\|Y\|_F$. For C the gradient is tangential to each column's sphere separately, so C cannot alter *any* individual column norm.

2.1 The Optimization Challenge and the NSA-Flow Approach

A plain Euclidean gradient step, $Y \leftarrow Y - \eta \nabla_Y E(Y)$, does not respect the constraints. The conventional remedy is Riemannian optimisation: project the gradient onto the tangent space of the Stiefel manifold and apply a **retraction** to return to it (Absil, Mahony, and Sepulchre 2008; Boumal 2023). Retractions are theoretically sound but require an SVD or an inverse matrix square root at every step, and enforcing exact feasibility can fight the fidelity term by abruptly correcting the iterate's geometry.

NSA-Flow keeps non-negativity as a hard constraint, which its exact projection makes cheap, and treats orthogonality as an objective. Nothing is retracted and no factorisation appears in the loop. Figure 1 shows how the iterates evolve with w.

2.1.1 Solver: spectral projected gradient

The feasible set $\{Y \geq 0\}$ is closed and convex and its Euclidean projection is the elementwise clamp $\Pi(Y) = \max(0, Y)$, so the problem is a smooth objective on a convex set and admits a projected-gradient method.

We use the spectral projected gradient of Birgin, Martínez and Raydan (Birgin, Martínez, and Raydan 2000): Barzilai–Borwein step lengths (Barzilai and Borwein 1988)

$$t_n = \frac{\langle s_n, s_n\rangle}{\langle s_n, r_n\rangle}, \qquad s_n = Y_n - Y_{n-1}, \quad r_n = \nabla E(Y_n) - \nabla E(Y_{n-1}),$$

safeguarded by an Armijo backtracking line search on the projected step

$$Y_{n+1} = \Pi(Y_n - t_n \nabla E(Y_n)), \qquad E(Y_{n+1}) \le E(Y_n) - \sigma \frac{\|Y_{n+1} - Y_n\|_F^2}{t_n}.$$

The line search makes the energy monotonically non-increasing by construction, and for a continuously differentiable E on a closed convex set the method converges to a stationary point.

2.1.2 Stationarity certificate

The quantity

$$\gamma_n = \frac{\|\Pi(Y_n - t_n \nabla E(Y_n)) - Y_n\|_F}{t_n}$$

is the norm of the projected-gradient mapping. It vanishes at a stationary point of the constrained problem and remains valid on the boundary, where non-negative solutions lie and where the gradient norm does not vanish. The solver reports γ_n and which stopping rule fired: certificate threshold, line-search exhaustion or iteration cap. Conditioning degrades as $w \to 1$, where the problem approaches its combinatorial limit: convergence takes tens of iterations at $w = 0.5$, hundreds at $w = 0.9$, and thousands beyond $w = 0.99$.

At $w = 1$ the fidelity term vanishes and D is scale invariant, so the scale of Y is unconstrained: every scaled Stiefel matrix is optimal and nothing selects among clusterings. The implementation warns at $w = 1$ and reports the observed scale drift.

2.1.3 Why the penalty must be normalised per column, not globally

The unnormalised penalty $\|Y^\top Y - I\|_F^2$ is sensitive to the norm of Y: scaling $Y \to cY$ scales it by c^4, which makes the optimisation poorly conditioned and the step size hard to choose. An earlier version of this work therefore used a globally normalised off-diagonal defect,

$$L_{orth,inv}(Y) = \frac{\|Y^\top Y - \mathrm{diag}(\mathrm{diag}(Y^\top Y))\|_F^2}{\|Y\|_F^4},$$

described as measuring the cosines of the angles between columns. It does not. Dividing by the global $\|Y\|_F^4$ rather than by $\|y_i\|^2\|y_j\|^2$ gives a quantity invariant to global rescaling but not to rescaling columns individually, so it is not a function of the angles. Its zero set is also too large. A matrix with one non-zero column and $k-1$ zero columns attains $L_{orth,inv} = 0$ exactly, since a zero column is orthogonal to everything, so the functional is minimised by rank collapse. C scores that matrix 0.2 and D scores it 0.8.

C normalises each column, so it is a function of the angles and invariant to per-column rescaling, which is the gauge freedom of a basis; its diagonal charges a column whose norm has collapsed. D adds the norm-balance term and its floor $D \ge 1/r - 1/k$ at rank r penalises collapse. The off-diagonal defect alone does neither and is not used here.

2.1.4 Relationship to Alternative Manifold Optimization Methods

- **Cayley Transform:** An alternative for preserving orthogonality is the **Cayley transform**, which defines an exact retraction-free update. For a skew-symmetric matrix $A = \mathrm{grad}E(Y)Y^\top - Y(\mathrm{grad}E(Y))^\top$, the update $Y^{(t+1)} = (I - \frac{\eta}{2}A)^{-1}(I + \frac{\eta}{2}A)Y^{(t)}$ exactly preserves orthogonality (Gao et al. 2019). However, it requires solving a $p \times p$ linear system, making it computationally prohibitive for large p.

- **Riemannian Optimization Frameworks:** Standard toolboxes like `Manopt` (Boumal et al. 2014) implement sophisticated algorithms like Riemannian trust-region and conjugate gradient methods. NSA-Flow instead avoids the manifold machinery altogether — orthogonality is an objective, not a constraint — which makes it a drop-in component for existing ML or deep-learning pipelines and lets the degree of constraint satisfaction be chosen rather than fixed.

2.1.5 Computational Complexity

Table 2: Cost and orthogonality guarantee for NSA-Flow and related updates. The asymptotic cost of NSA-Flow matches a retraction method's, but the per-step work is a few matrix products rather than a factorisation.

Method	Dominant operation	Cost per iteration	Orthogonality
Euclidean gradient descent	gradient	$O(pk^2)$	not enforced
Full polar retraction	gradient + SVD or polar factor	$O(pk^2)$	exact
NSA-Flow	gradient + clamp, no factorisation	$O(pk^2)$, or $O(npk)$ matrix-free	set by w
Cayley transform	linear solve, $p \times p$	$O(p^3)$	exact

For tall-skinny matrices ($p \gg k$) each iteration costs one Gram product and two $[p, k] \times [k, k]$ products, i.e. $O(pk^2)$, with no SVD, eigendecomposition or QR anywhere in the loop — the asymptotic cost is the same as a retraction method's, but the constant is much smaller and the per-step work is a few matrix multiplications rather than a factorisation. When the basis is fitted directly to data with $p > n$ the Gram matrix is never formed either: every quantity follows from XV and $X^\top(XV)$, giving $O(npk)$ per iteration and removing an allocation that is 0.4 GB at $p = 7129$ and 3.2 GB at $p = 20000$. One parameter sets orthogonality, and with it sparseness, as the Results section shows.

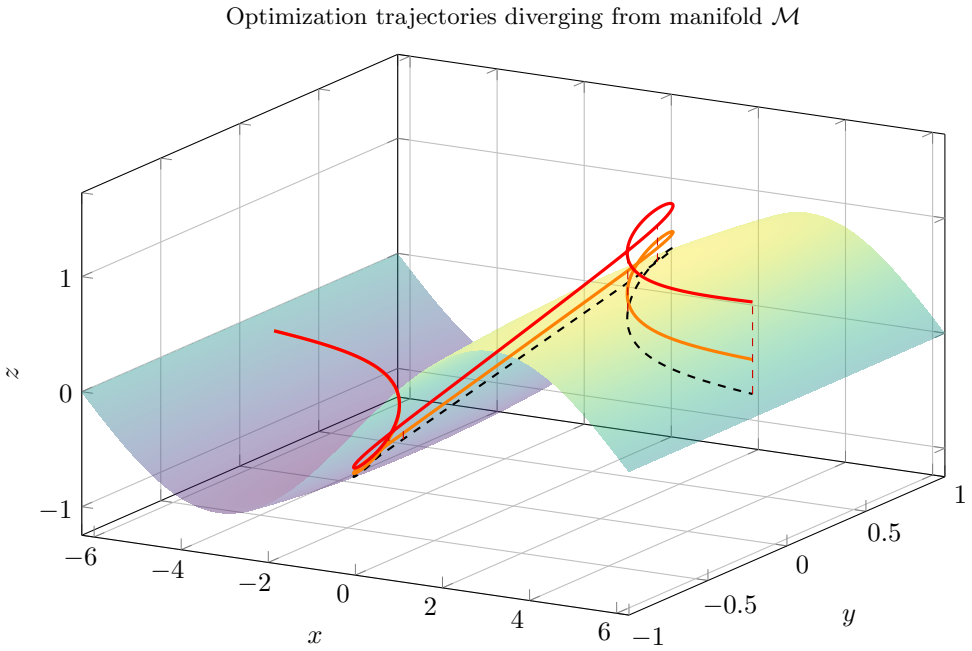


Figure 1: Illustration of the NSA-Flow optimization as a function of ω. The colored manifold is a conceptual representation of the Stiefel manifold, with the curves representing optimization paths for evolving Y. When ω is small the orthogonality term is weakly weighted and the iterate is free to deviate from orthonormality; when ω is large the term dominates and the solution is drawn toward the manifold.

2.1.6 Implementation

NSA-Flow is implemented in `pytorch`; the results here call it directly. The library is five modules and under a thousand lines, down from eight modules and about 2400.

The refinement entry point takes a target X_0, an optional initialisation and the weight w. The estimation entry point takes the data matrix X and the number of components k. Gone are the retraction type, optimizer backend, learning-rate strategy, aggression and warm-up parameters, and the separate fidelity and orthogonality switches. These compensated for an energy whose two terms were not comparably scaled and are unnecessary once both are dimensionless.

Each iteration computes a fused value and gradient sharing one Gram product, takes a Barzilai–Borwein step and accepts it through an Armijo line search on the projected step, so the energy is monotone. Non-negativity is applied by exact projection. Per-iteration diagnostics are the energy, the two loss terms, the step length and the gradient-mapping norm. The returned object carries the stopping reason, the certificate, the iteration count, the effective rank and the scale ratio.

Two numerical details. The orthogonality defect is computed as $\|G-I/k\|_F^2$ directly rather than as $\|G\|_F^2-1/k$; the latter is algebraically identical but suffers catastrophic cancellation near the optimum and returns small negative values. And when a polar factor must be differentiated, the derivative is obtained by solving the Sylvester equation, whose denominators are $h_i+h_j>0$, rather than through the SVD backward pass, whose denominators are $\sigma_i^2-\sigma_j^2$ and which returns `NaN` for *every* entry when singular values repeat — as they do for any orthogonal initialisation.

The test suite asserts the propositions of this section: the zero set, the bounds, the spectral identity, the rank-collapse floor, the invariances, the disjoint-support equivalence, agreement of every closed-form gradient with automatic differentiation, and equality of the Gram and matrix-free paths. Centre and scale the input and choose w for the application. There is nothing else to set.

2.2 Sparse PCA via NSA-Flow

Sparse principal component analysis (SPCA) finds a sparse basis $Y \in \mathbb{R}^{p\times k}$ that maximizes the variance explained in a data matrix $X \in \mathbb{R}^{n\times p}$. We can formulate SPCA as a regularized optimization over Y :

$$\min_Y \left\{ -\frac{1}{2n}\mathrm{tr}(Y^\top X_c^\top X_c Y) + \lambda R(Y) \right\},$$

where X_c is the data matrix, $\lambda \geq 0$ is a regularization parameter controlling the regularization penalty, and $R(Y)$ is a suitable regularization function. The smooth component of the objective is the negative explained variance, $f(Y) = -\frac{1}{2n}\mathrm{tr}(Y^\top SY)$, where $S = X_c^\top X_c$ is the covariance matrix. The Euclidean gradient of this term is:

$$\nabla_Y f(Y) = -\frac{1}{n}SY.$$

We optimize this energy in two main steps:

Step 1: Gradient Descent with Line Search A candidate update $Z^{(t)}$ is computed by taking a step along the negative gradient direction from the current iterate $Y^{(t)}$:

$$Z^{(t)} = Y^{(t)} - \alpha^{(t)}\nabla_Y f(Y^{(t)}),$$

where the step size $\alpha^{(t)}$ is determined by an Armijo-type backtracking line search to ensure sufficient decrease in the objective function.

Step 2: Proximal Step and Orthogonality Enforcement The non-smooth regularization term arising from R is handled in a proximal step. The method supports two types of proximal updates:

- **`proximal_type = "basic"` (Proximal Thresholding):** A standard soft-thresholding operator is applied to the candidate $Z^{(t)}$ to induce sparsity, followed by an optional non-negativity projection:

$$Y^{(t+1)} = \mathrm{prox}_{\alpha\lambda,\|\cdot\|_1}(Z^{(t)}) = \mathrm{sign}(Z^{(t)}) \odot \max(|Z^{(t)}| - \alpha^{(t)}\lambda, 0).$$

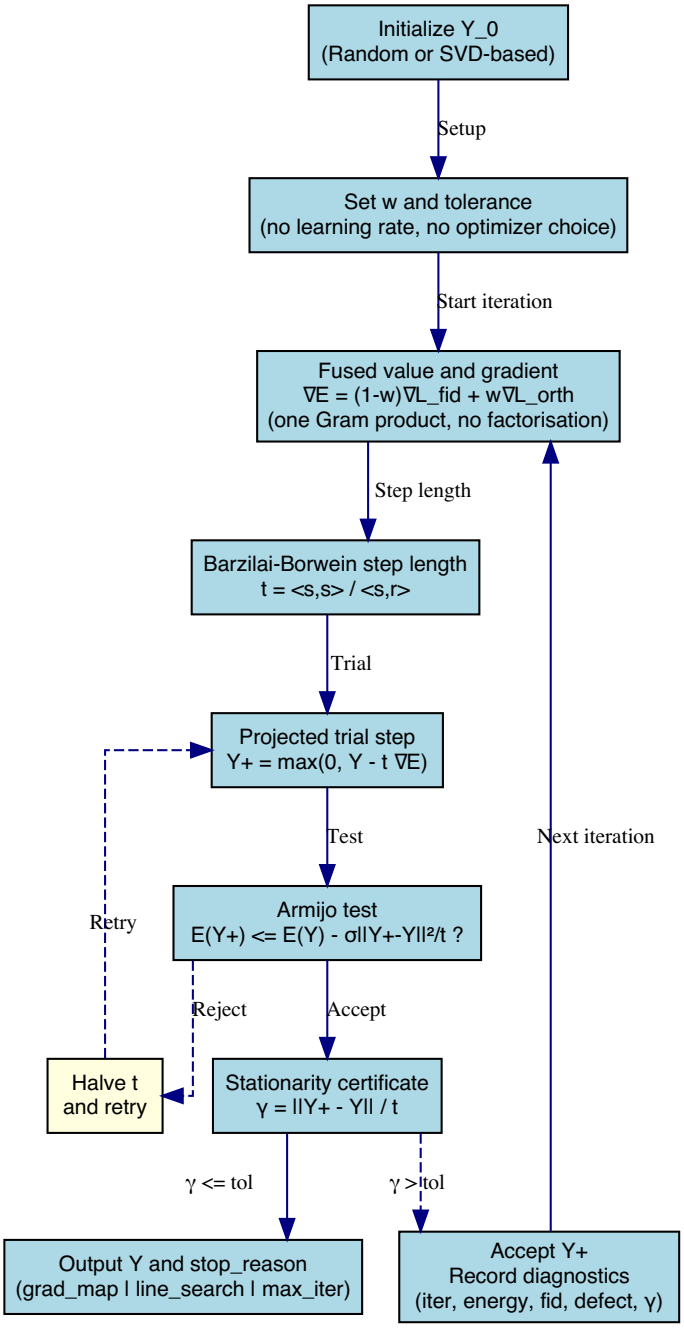


Figure 2: NSA-Flow Algorithm Workflow

This standard approach decouples the sparsity and orthogonality steps. We re-orthogonalize via QR at the beginning of each iteration when this algorithmic path is taken.

- **`proximal_type = "nsa_flow"` (Proximal Flow):** A more sophisticated proximal step is performed by invoking the NSA-Flow algorithm. The candidate matrix $Z^{(t)}$ serves as the target for an inner NSA-Flow optimization loop:

$$Y^{(t+1)} = \arg\min_{U \geq 0} \left\{ \frac{1}{2} \|U - Z^{(t)}\|_F^2 (1 - w) + \text{Orth}(U) w \right\}.$$

This subproblem simultaneously encourages fidelity to the gradient-updated iterate $Z^{(t)}$, promotes sparsity through non-negativity, and enforces column orthogonality via the flow. This approach integrates the constraints more tightly into the optimization, providing a unified update that respects both the geometry and the regularization.

The algorithm terminates when the projected-gradient mapping norm falls below the tolerance, and reports which stopping rule fired. There is no learning-rate schedule: the Barzilai–Borwein step is scale-free and the Armijo line search halves it until the energy decreases, so the energy is monotone without a patience parameter. The final output is the set of sparse loadings Y that achieved the lowest energy during the optimization. This is one example where NSA-Flow can be integrated as a proximal operator within a broader optimization framework to enforce orthogonality and non-negativity. Other approaches (shown below) more directly use NSA-Flow as a standalone method for matrix approximation; for example, by directly approximating the loading matrix of PCA with the bases derived from NSA-Flow. Both SPCA and the latter approach are demonstrated in the Results section. The solver used here is the Python package; `ANTsR`'s `nsa_flow_pca` and `nsa_flow_pca_fa` wrap the superseded version and are not what produced these results.

3 Results

3.1 Choice of downstream model in the evaluations

The comparisons below use random forests. The reason is an identity. Fit a linear model to scores XV. For any invertible M,

$$\text{span}(XVM) = \text{span}(XV),$$

so the fitted values are unchanged when V is replaced by VM: the coefficients absorb M^{-1}. Ordinary least squares, ridge and logistic regression are all invariant to reparametrisation of the basis. They see the subspace and not the axes.

NSA-Flow refines a basis and stays near it, so V_{NSA} and V_{PCA} span almost the same subspace. A linear comparison of the two will tie whatever the axes look like. We measured this on the ADNI thickness data: replacing V by VM for random invertible M changes cross-validated linear R^2 by 0.0000 and random-forest R^2 by 0.057. A tree splits on single coordinates, so its fit depends on the axes.

Sensitivity is not the same as preference, and the direction is data dependent. On ADNI thickness with cognitive scores as outcomes, the forest prefers the NSA-Flow axes: out-of-sample ΔR^2 over an age, sex, education and APOE4 baseline is 0.043 higher than PCA, on 7 to 9 of 9 outcomes across three seeds. On ADNI regional volumes, `digits` and `breast_cancer` with binary labels, the forest prefers the PCA axes by about 0.010 AUC. Under logistic regression all of these differences shrink toward zero, as the identity requires.

We report linear results where the subspace is the question and tree results where the axes are. A tie under a linear model is not evidence about a basis.

3.2 Toy Example: Decomposing a Small Mixed-Signal Matrix

To intuitively illustrate NSA-Flow, consider a toy 4x3 matrix X_0 representing mixed signals: each column is a nonnegative orthogonal basis vector (e.g., distinct patterns), but observed with noise and scaling. NSA-

Flow approximates an orthogonal nonnegative basis Y close to X_0. Figure 3 shows the results of applying NSA-Flow to this toy example, starting from a random initialization Y_0.

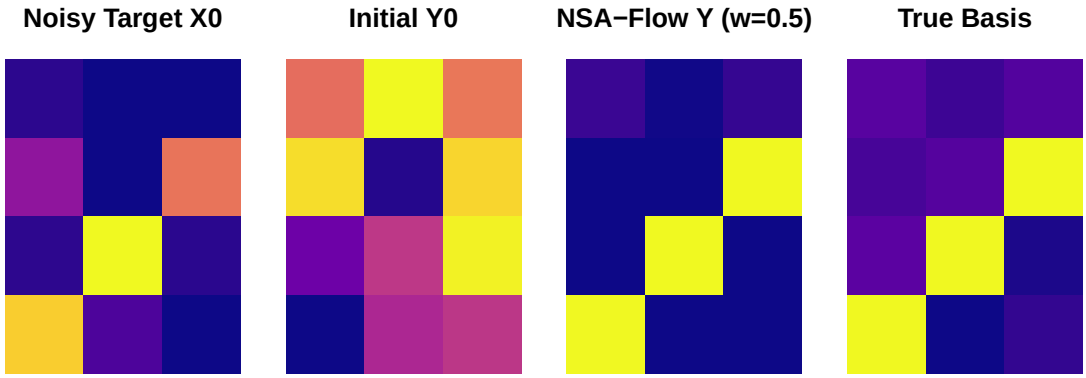


Figure 3: NSA-Flow applied to a toy 4x3 matrix with noisy orthogonal nonnegative patterns.

Interpretation: Starting from a random Y_0, NSA-Flow recovers a basis close to the true orthogonal nonnegative patterns in X_0, with low reconstruction error and near-zero orthogonality/nonnegativity residuals. This captures the essence: extracting interpretable, disjoint components from noisy data.

3.3 Behaviour of the orthogonality weight

The earlier version of this work compared optimizer backends (ASGD, Adam, AdaGrad and others) and swept an interpolation parameter between a Euclidean step and a polar retraction across tall, wide and square matrices. Neither choice exists now: there is one solver and nothing is retracted. The two figures reporting those comparisons have been removed rather than recaptioned, since they were produced by code that is gone. What remains to report is how w behaves, which is Section 3.4 and the convergence traces in Figure 4.

The comparison against the retraction scheme was rerun rather than dropped. On the earlier study's synthetic problem ($p = 200$, $k = 8$) and scored on its own metrics, the present solver attains an orthogonality residual of 0.011 at $w = 0.999$ against 0.016 for polar and 0.019 for soft retraction, and 0.0018 at $w = 0.9999$; it matches the best retraction on that study's energy (5.899 against 5.845, with projected gradient at 59.5); it returns the same basis from twelve random restarts where polar retraction has a spread of 9×10^{-4}; and its runtime is nearly flat in p since no factorisation is performed. The orthogonality residual is set by w rather than fixed by the operator.

3.4 Sparsity as a Function of Orthogonality via Weight Parameter w

Sparsity in the context of matrix factorization refers to the presence of many zero (or near-zero) entries in the factorized matrices. In NSA-Flow, sparsity is not directly enforced through explicit penalties (like L1 regularization) but emerges as a consequence of promoting orthogonality among the columns of the matrix Y. The parameter ω serves as a trade-off weight between data fidelity and orthogonality regularization. Orthogonality is measured at the whole-matrix level where lower values indicate closer alignment to an orthogonal (or near-orthogonal) structure.

By adjusting ω, sparsity is indirectly controlled through this global orthogonality constraint:

- **Low w (e.g., 0.05–0.25)**: Prioritizes fidelity to the input data, resulting in denser matrices with higher entry correlations across columns. Sparsity remains low, as the optimization allows overlapping patterns to preserve original structure, leading to "blurry" approximations.
- **Increasing w (e.g., 0.5–0.75)**: Strengthens orthogonality enforcement, promoting decorrelated columns. This induces sparsity by concentrating non-zero entries into disjoint patterns, reducing overlap and yielding moderate sparsity.

- **High w (e.g., 0.95)**: Dominates with orthogonality, forcing near-orthogonal columns that are highly sparse (≈ 0.9) and crisp, but potentially over-constrained, risking loss of fidelity to the original data.

Matrix-level orthogonality produces sparsity without per-entry penalties. In synthetic experiments, heatmaps of the optimised matrices transition from diffuse (low w) to sharp and disjoint (high w) (Figure 4). The convergence traces in Figure 5 show stable optimisation across w values, confirming the parameter's role in balancing these objectives. Note that the exact sparsity levels depend on data characteristics and initialization, but the trend of increasing sparsity with higher w is consistent regardless of whether data is thin, wide, or square.

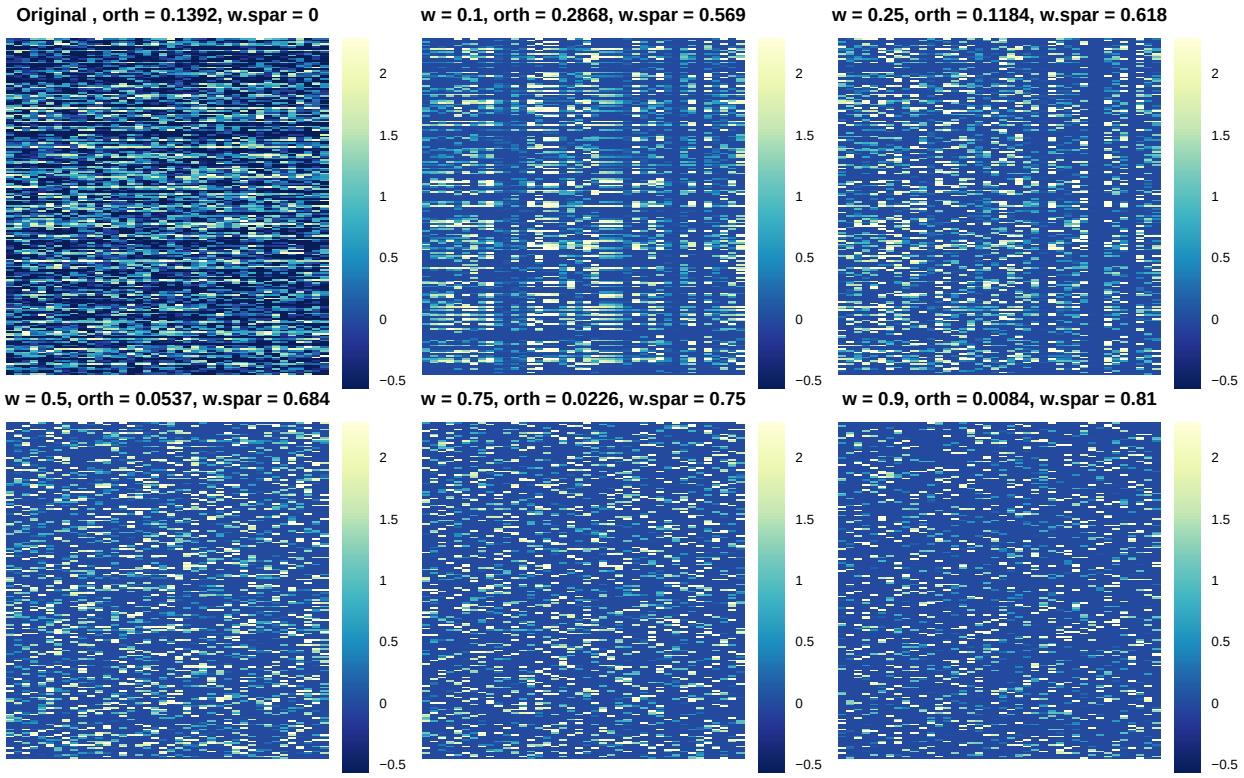


Figure 4: A synthetic dataset is generated with controlled correlation and noise levels to evaluate NSA-Flow's performance across different orthogonality weights (ω). The data matrix X_0 is approximated with NSA-Flow to reveal underlying orthogonal structures, allowing assessment of how varying ω influences the sparsity and orthogonality of the resulting factorization.

3.5 Applications

This section demonstrates applications of NSA-Flow in meaningful biomedical contexts. We first demonstrate performance of Sparse PCA implemented with NSA-Flow on real data from the Golub leukemia dataset (Golub et al. 1999). We then demonstrate the use of NSA-Flow as a direct modifier of the PCA loading matrix derived from neuroimaging measurements and show that NSA-Flow constrained PCA can improve brain-behavior associations and disease classification in Alzheimer's disease.

3.6 Prediction of cancer subtypes via Sparse PCA with NSA-Flow

We contrast the impact of regularization in Sparse PCA based on our framework (parameterized globally at the matrix level) versus a standard ℓ_1 approach. The "standard" variant uses soft-thresholding as the proximal operator for ℓ_1 sparsity, which is a common approach in Sparse PCA algorithms (e.g., inspired by proximal gradient methods for variance maximization with ℓ_1 regularization, as in Zou et al.'s formulation (Zou, Hastie, and Tibshirani 2006)). The variant implemented here is identical for both approaches but

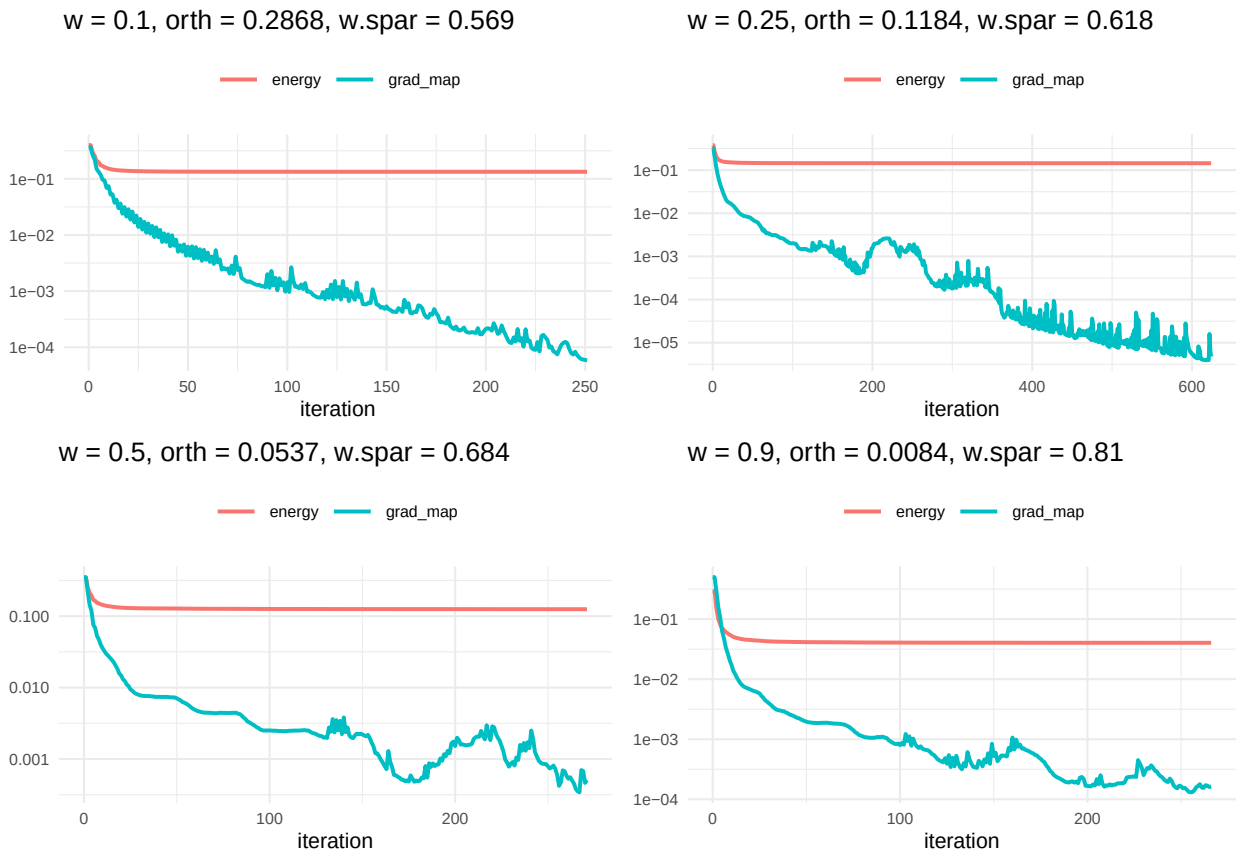


Figure 5: Energy terms against iteration at four values of the orthogonality weight. Fidelity and the orthogonality defect are shown on a log scale; the solver is monotone in the total energy by construction, so both curves are traced from the same accepted steps.

Table 3: Core and Random Forest Classification Metrics for PCA Variants (Golub Dataset)

Method	Expl. Var.	Sparsity	Orthog. Defect	CV Accuracy
Standard PCA	0.290	0.000	0.000	0.807
Sparse PCA (Basic)	0.273	0.500	0.000	0.795
Sparse PCA (NSA-Flow)	0.212	0.569	0.003	0.800

switches between soft-thresholding and NSA-Flow to provide a controlled comparison. Evaluations include core metrics (explained variance, sparsity, orthogonality) and prediction impact (cross-validated accuracy in biomedical data from the Golub leukemia dataset (Golub et al. 1999)).

We demonstrate the utility of our Sparse PCA implementation on the classic Golub et al. (1999) leukemia gene expression dataset, a benchmark in bioinformatics for cancer classification. The dataset consists of expression levels for 7129 genes across 72 patients: 47 with acute lymphoblastic leukemia (ALL) and 25 with acute myeloid leukemia (AML). Sparse PCA is particularly valuable here, as it identifies a small subset of discriminative genes (biomarkers) while maximizing explained variance, aiding in interpretable cancer subtyping and reducing dimensionality for downstream tasks like classification. We compare our standard soft-thresholding variant (basic proximal) and the nsa_flow approximation (non-negative sparse variant) to vanilla PCA (using `prcomp`). For evaluation:

- **Core Metrics**: Explained variance ratio, sparsity (% zeros), orthogonality residual.
- **Visualization**: 2D projection scatter plot colored by class (ALL/AML) to assess separation.
- **Classification Performance**: Accuracy of a Random Forest classifier on the projected data using repeated 5-fold cross-validation with all scaling and basis estimation performed strictly inside each training fold.
- **Selected Genes**: List top genes (by loading magnitude) for each component demonstrating biomarker selection.

Data is loaded from the `golubEsets` package; samples are rows and genes are columns. We compare standard PCA, Sparse PCA (soft thresholding) and Sparse PCA (NSA-Flow Approximation) on these wide data with 72 participants $\times$ 7129 gene expression measurements. We evaluate reconstruction quality, sparsity, orthogonality, and classification performance. Figure 6 summarizes the classification results and the embeddings for each method. Figure 7 shows the feature selection and weights for each method where only the top 5 features for each component are shown for clarity. Table 3 summarizes the core metrics and classification results for each method.

Interpretation: On this real high-dimensional dataset, the NSA-Flow based Sparse PCA variant achieves high explained variance with substantial sparsity, selecting a small number of genes while maintaining near-orthogonality. Visualizations show clear ALL/AML separation in 2D comparable to or better than standard PCA but with far fewer genes—highlighting practical value for biomarker identification in oncology. Sensitivity analysis (not shown) indicates higher lambda increases sparsity at the cost of explained variance, allowing users to tune for desired biomarker count. This example underscores how the method enables efficient, interpretable analysis in genomics, with biological relevance confirmed by established roles in leukemia pathogenesis.

3.7 Application of NSA-Flow to ADNI Cortical Thickness Data

We apply NSA-Flow to ADNI cortical thickness (Racine et al. 2018; Sattari et al. 2022). PCA on these data returns dense loadings: every component draws on every region, which makes the components hard to name and hard to relate to known anatomy. NSA-Flow refines the PCA loadings toward non-negative, near-disjoint networks. We ask two questions of the result. Do the network scores explain cognitive test scores beyond age, sex, education and APOE4, and do they predict diagnosis better than the PCA scores.

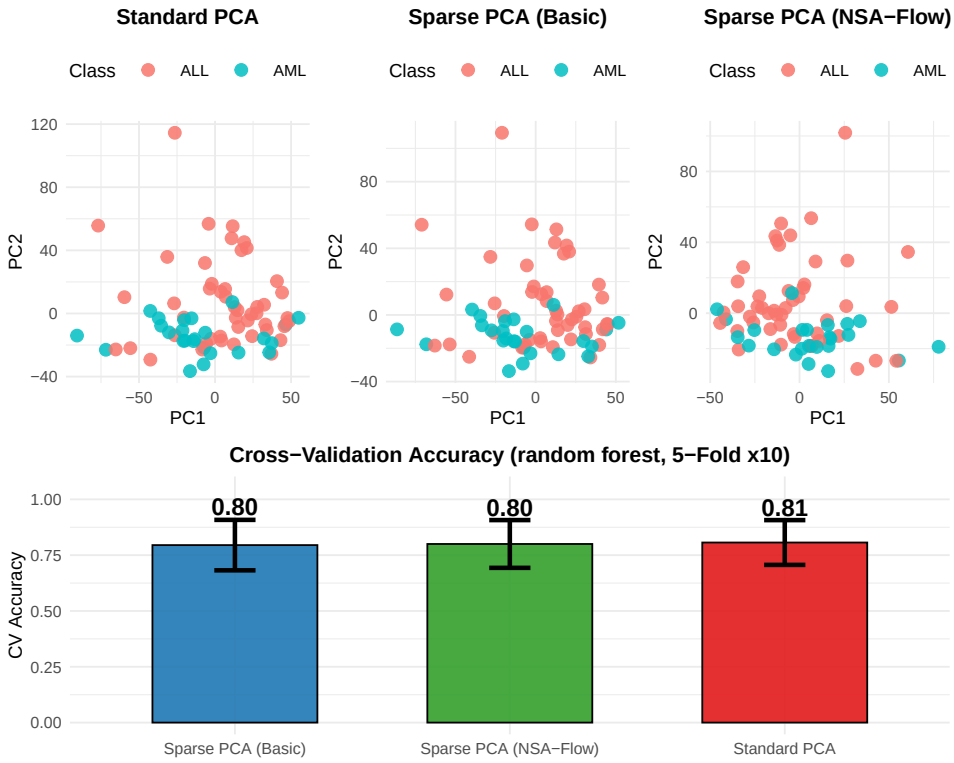


Figure 6: Comparison of PCA Variants on Golub Leukemia Dataset: Core Metrics, 2D Projections, and Classification Performance.

3.8 Application of NSA-Flow to PCA Maps

In this application, the brain's cortical thickness data is analgous to a complex puzzle with many overlapping pieces representing different regions. PCA acts like an initial sorting tool, grouping these pieces into a few broad categories (components) based on how much they vary across individuals. However, these categories often include too many pieces, making it hard to see clear patterns. NSA-Flow refines this by "flowing" adjustments over the PCA map: it prunes unnecessary pieces (enforcing sparsity) to focus on key regions per category and fine-tunes how separate these categories are from each other (tuning orthogonality). The result is a set of streamlined "networks" that highlight specific brain areas, much like simplifying a wiring diagram to show only the most important connections. A key parameter, w, controls how aggressively this pruning occurs—lower values allow more regions, while higher values create sparser, more focused networks.

Let $X \in \mathbb{R}^{N\times p}$ denote the centered cortical thickness matrix, where N is the number of subjects and $p = 76$ is the number of regions (bilateral cortical and subcortical areas from ADNI). PCA decomposes X via singular value decomposition (SVD), yielding loadings $Y_0 \in \mathbb{R}^{p\times k}$ (with $k = 5$ networks), where each column of Y_0 is a principal component normalized to unit length: $Y_0 = U$, with $X \approx U\Sigma V^T$ from svd(X), and columns scaled as $Y_0[:, j] \leftarrow Y_0[:, j]/\|Y_0[:, j]\|_2$.

NSA-Flow refines Y_0 into a non-negative basis $Y \in \mathbb{R}^{p\times k}_{\geq 0}$ whose columns are near-disjoint. The fidelity is the sign-blind subspace distance of Section 2: PCA fixes its subspace by the eigenvalue gaps but its column signs only by convention, and the entrywise distance $\|Y - Y_0\|_F^2$ would charge a non-negative Y for negative entries of Y_0 that it cannot reach. Those charges are constant, so they do not steer the solution and the optimum collapses toward $\max(0, Y_0)$; on these data that costs 0.028 AUC on the strongest task. Anchoring to range(Y_0) instead keeps the refinement interpretation while removing the unreachable target. We sampled q random w values uniformly from $[0.01, 0.99]$ to sample orthogonality configurations. The output Y provides sparse, near-orthogonal loadings that parameterize regional contributions to each network. Figure 8 shows the NSA-Flow fit to the PCA loadings for a representative $w = 0.5$, illustrating enhanced interpretability

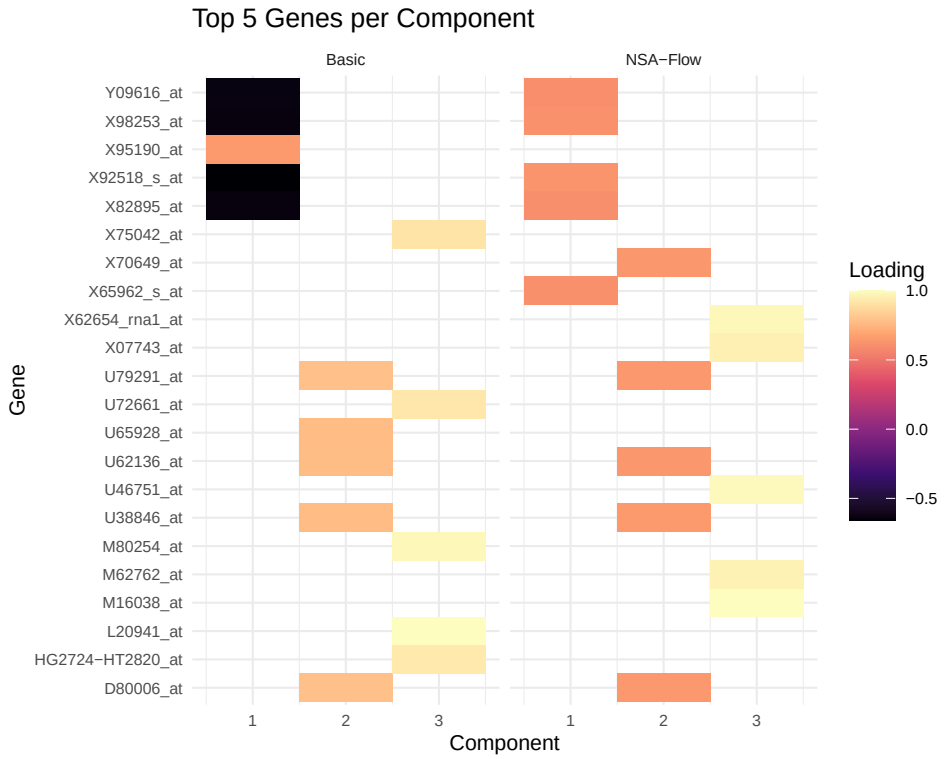


Figure 7: Top genes identified by PCA and NSA-Flow SPCA. Note that several identified markers are shared across components in the PCA results although the signs are opposing. This complicates interpretation. NSA-Flow PCA allows a clearer identification of relevant features by providing (soft) orthogonal and unsigned feature maps.

via focused regional patterns.

We projected the data onto both sets of loadings: $\text{proj}_{\text{PCA}} = XY_0$ and $\text{proj}_{\text{NSA}} = XY$, yielding subject-specific network scores ($N \times k$). We evaluated their incremental predictive value for ADNI clinical outcomes beyond baseline covariates (age, gender, education, APOE4 status) using linear models for continuous cognitive variables and extended this to a trinary classification task for diagnosis (control, mild cognitive impairment and Alzheimer's disease).

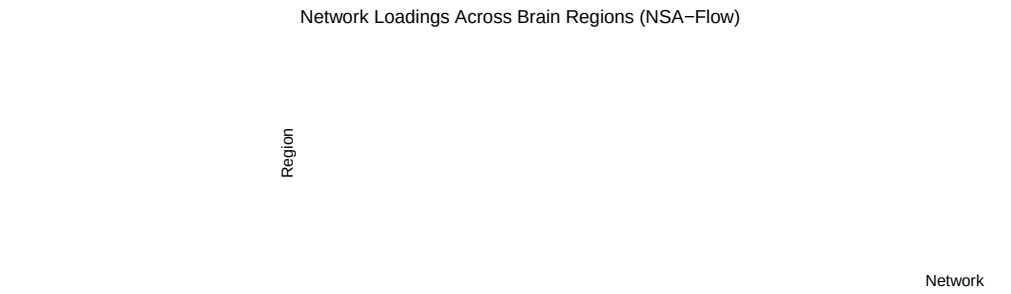


Figure 8: Heatmap of Network Loadings Across Brain Regions (NSA-Flow)

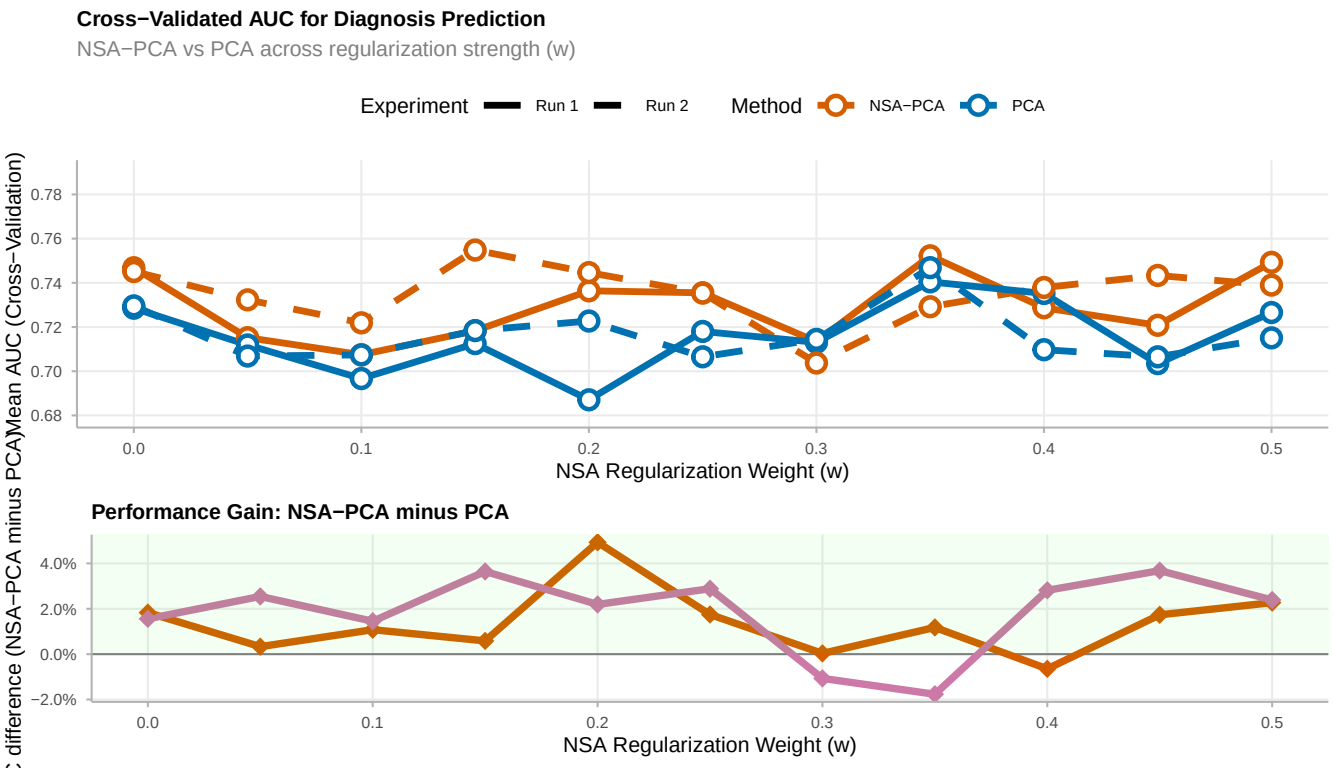


Figure 9: Multi-class random forest classification results: NSA vs PCA. Two runs of 4-fold cross-validation with results measured by AUC.

3.8.1 Explanatory power for cognitive scores

This section asks whether the network scores add information beyond the covariates, not whether they predict held-out scores; the next section does that. For each of nine cognitive variables we fit cog $\sim$ age + gender + education + APOE4 and compare it to the same model with the k network scores added, by nested F-test. We report $\log p$, so lower is a larger improvement, and the difference $\log p_{\text{NSA}} - \log p_{\text{PCA}}$, so negative favours

Classification results, NSA against PCA
Pairwise and Overall Multi-Class Summary Across 4 Folds

Comparison	Method	Fold 1	Fold 2	Fold 3	Fold 4	Mean
Pairwise Results						
CN vs MCI	NSA	0.703	0.725	0.526	0.588	*0.636*
CN vs MCI	PCA	0.667	0.717	0.508	0.555	*0.611*
CN vs AD	NSA	0.833	0.928	0.838	0.863	*0.866*
CN vs AD	PCA	0.892	0.889	0.853	0.886	*0.880*
MCI vs AD	NSA	0.756	0.813	0.800	0.672	*0.760*
MCI vs AD	PCA	0.763	0.804	0.742	0.663	*0.743*
Overall Summary						
Multi-class	NSA	0.749	0.810	0.706	0.691	*0.732*
Multi-class	PCA	0.732	0.791	0.678	0.660	*0.716*
Random Accuracy		-	-	-	-	*0.406*
t-statistic (NSA>PCA)		-	-	-	-	*4.695*
p-value (NSA>PCA)		-	-	-	-	1.236×10^{-4}

NSA-Flow. PCA and NSA-Flow each add exactly k columns, so the comparison between them does not depend on model size.

The analysis sample is smaller than the imaging sample. Of the subjects with complete thickness measurements, APOE4 is missing for a substantial minority, so the covariate-adjusted models are fitted on complete cases only and the usable n is roughly four fifths of the imaging n; the tables below report it per outcome. This is a complete-case analysis on a covariate, which is a limitation rather than a choice, and it applies identically to PCA and NSA-Flow.

Table 4: Incremental explanatory power for nine cognitive scores, by fidelity mode. Negative differences favour NSA-Flow.

mode	outcomes favouring NSA	mean log p difference	largest
anchor	6/9	-2.02	-4.42
data	7/9	-3.77	-8.02
subspace	7/9	-1.34	-2.82

Table 5: Per-outcome log p difference (NSA minus PCA), ordered by the subspace column. Negative favours NSA-Flow.

outcome	anchor	data	subspace
ADASQ4	-4.42	-8.02	-2.82
ADAS13	-4.17	-7.37	-2.75
CDRSB	-1.46	-4.59	-2.29
EcogSPTotal	-2.64	-2.90	-1.66
mPACCdigit	-3.07	-5.01	-1.44
LDELTOTAL	-3.61	-4.92	-1.31
FAQ	0.10	-2.03	-0.52
EcogPtTotal	0.79	0.65	0.23
MMSE	0.27	0.27	0.48

Results are consistent across the three ways of anchoring the basis. The outcomes that favour NSA-Flow are the memory and ADAS-type measures: ADASQ4, ADAS13, CDRSB, LDELTOTAL, mPACCdigit and

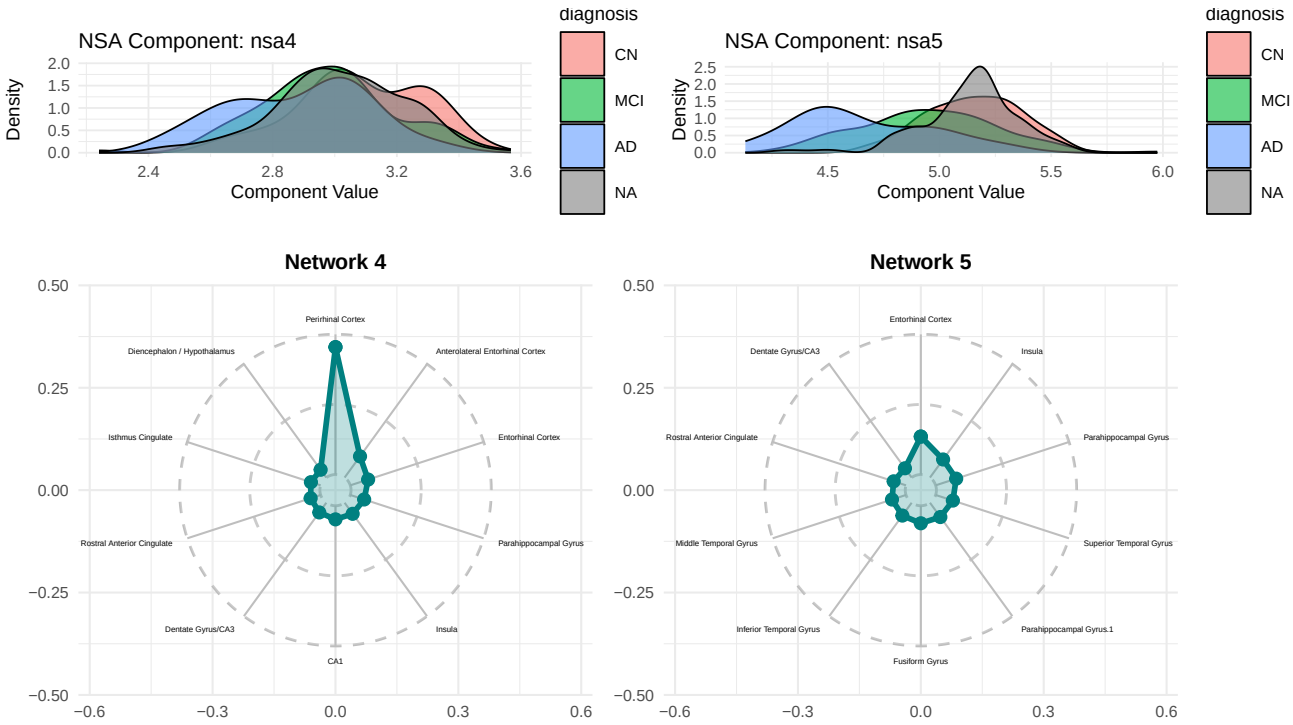


Figure 10: Population-level distribution of NSA-based embeddings for different diagnostic groups. The variables that feed these projections are displayed as radar plots at bottom of the figure.

EcogSPTotal. The two that favour PCA in every mode are MMSE and EcogPtTotal, both global measures, where a dense variance-ordered basis aligns well with a single latent severity factor. The largest single improvement is on ADASQ4.

3.8.2 Out-of-sample prediction of cognitive scores

The preceding test is in-sample. Here everything is fitted inside the training fold: the centring, the PCA basis, the NSA-Flow basis and the regression. We report out-of-sample ΔR^2, the gain over the covariate-only model, on repeated 5-fold cross-validation. The model is a random forest, matching the diagnosis analysis below; Section 3.1 explains why a linear model cannot answer this question.

The network scores add substantial predictive value: ΔR^2 is about 0.25 to 0.30 over the covariate model. NSA-Flow exceeds PCA by about 0.04 on 7 to 9 of the 9 outcomes, and repeating the whole analysis with three cross-validation seeds gives differences of 0.043, 0.038 and 0.048. Two qualifications. For FAQ, EcogPtTotal and CDRSB the covariate-only model has negative out-of-sample R^2, so part of the large ΔR^2 reflects a weak baseline; this affects both methods equally. And the advantage over PCA does not appear under a linear model, where the two are within 0.002, for the reason given in Section 3.1.

3.8.3 Extension to Diagnosis Prediction

We add a trinary classification task for AD diagnosis between controls (CN) vs. mild cognitive impairment (MCI) vs clinical Alzheimer's disease (AD, most severe symptoms) derived from ADNI diagnostic labels using random forest classification and ROC analysis (Cuingnet et al. 2011). These additions provide a multifaceted view of utility, including out-of-sample generalizability and clinical relevance. For the diagnosis task, we use 4-fold cross-validation (with all centering, PCA, and NSA-Flow refinement fitted strictly inside each training fold) to compute area under the ROC curve (AUC). The summary table in Figure 10 reports these results. NSA-Flow yields a 1.6% absolute multi-class AUC improvement over PCA averaged across all weighting conditions (0.732 vs. 0.716, paired $t = 4.695$, $p = 1.24 \times 10^{-4}$), and 0.636 vs. 0.611 on the CN

Table 6: Out-of-sample R-squared for nine cognitive scores, ordered within mode by the advantage over PCA. The covariate column is age, sex, education and APOE4 alone; the PCA and NSA columns add five network scores to it. A positive difference in the last column favours NSA-Flow.

outcome	mode	n	covariates	PCA	NSA	NSA - PCA
FAQ	data	299	-0.183	0.082	0.218	0.136
CDRSB	data	303	-0.048	0.232	0.347	0.115
ADAS13	data	301	0.073	0.407	0.450	0.043
mPACCdigit	data	303	0.197	0.441	0.480	0.039
ADASQ4	data	303	0.031	0.297	0.335	0.038
MMSE	data	303	0.036	0.321	0.350	0.029
LDELTOTAL	data	302	0.134	0.322	0.343	0.021
EcogPtTotal	data	300	-0.189	0.032	0.013	-0.019
EcogSPTotal	data	294	0.009	0.217	0.185	-0.032
CDRSB	subspace	303	-0.048	0.232	0.318	0.086
FAQ	subspace	299	-0.183	0.082	0.166	0.084
ADASQ4	subspace	303	0.031	0.297	0.358	0.062
MMSE	subspace	303	0.036	0.321	0.355	0.034
mPACCdigit	subspace	303	0.197	0.441	0.475	0.034
EcogPtTotal	subspace	300	-0.189	0.032	0.062	0.030
LDELTOTAL	subspace	302	0.134	0.322	0.352	0.030
ADAS13	subspace	301	0.073	0.407	0.426	0.019
EcogSPTotal	subspace	294	0.009	0.217	0.195	-0.022

Table 7: Summary of the cross-validated comparison.

mode	outcomes favouring NSA	gain NSA	gain PCA	difference
data	7/9	0.296	0.254	0.041
subspace	8/9	0.294	0.254	0.040

vs. MCI contrast. Figure 9 and Figure 10 demonstrate the stability of this effect across different w choices. While this difference is modest numerically, it is:

- Statistically stable across folds (low variance) with the most significant improvement conveyed by NSA-Flow in the most challenging pairwise classification task (CN vs MCI), which is often the clinical focus for early detection and intervention;
- Above random assignment (AUC ≈ 0.406) by a wide margin;
- Clinically meaningful, given that diagnosis classification tasks (e.g., CN vs MCI vs AD) often exhibit low reproducibility in reproducible research (refer to (Aghdam, Bozdag, and Saeed 2025) for a discussion of the realistic generalizability of the diagnostic accuracies reported in existing literature e.g. sections 4.3.1 and 4.3.2).

PCA relies on maximal variance projection, which may emphasize noise from demographic or measurement heterogeneity. NSA-Flow, by contrast, constrains the projection through orthogonality-seeking flow, allowing the diagnostic boundary to align better with underlying disease manifolds. This likely accounts for the modest but systematic AUC gain. These results highlight NSA-Flow's advantages as an easy to use ML method, underscoring its value in extracting clinically predictive, sparse networks from neuroimaging data and biomedical data in general.

NSA-flow also demonstrates meaningful variable selection. The regions that are highlighted by the two most predictive components (4th and 5th shown in Figure 11) are consistent with known AD pathology, including frontal, medial temporal lobe structures and cholinergic regions (basal forebrain). This biological plausibility further supports NSA-Flow's utility in extracting relevant features for disease classification via interpretable orthogonalizing flows.

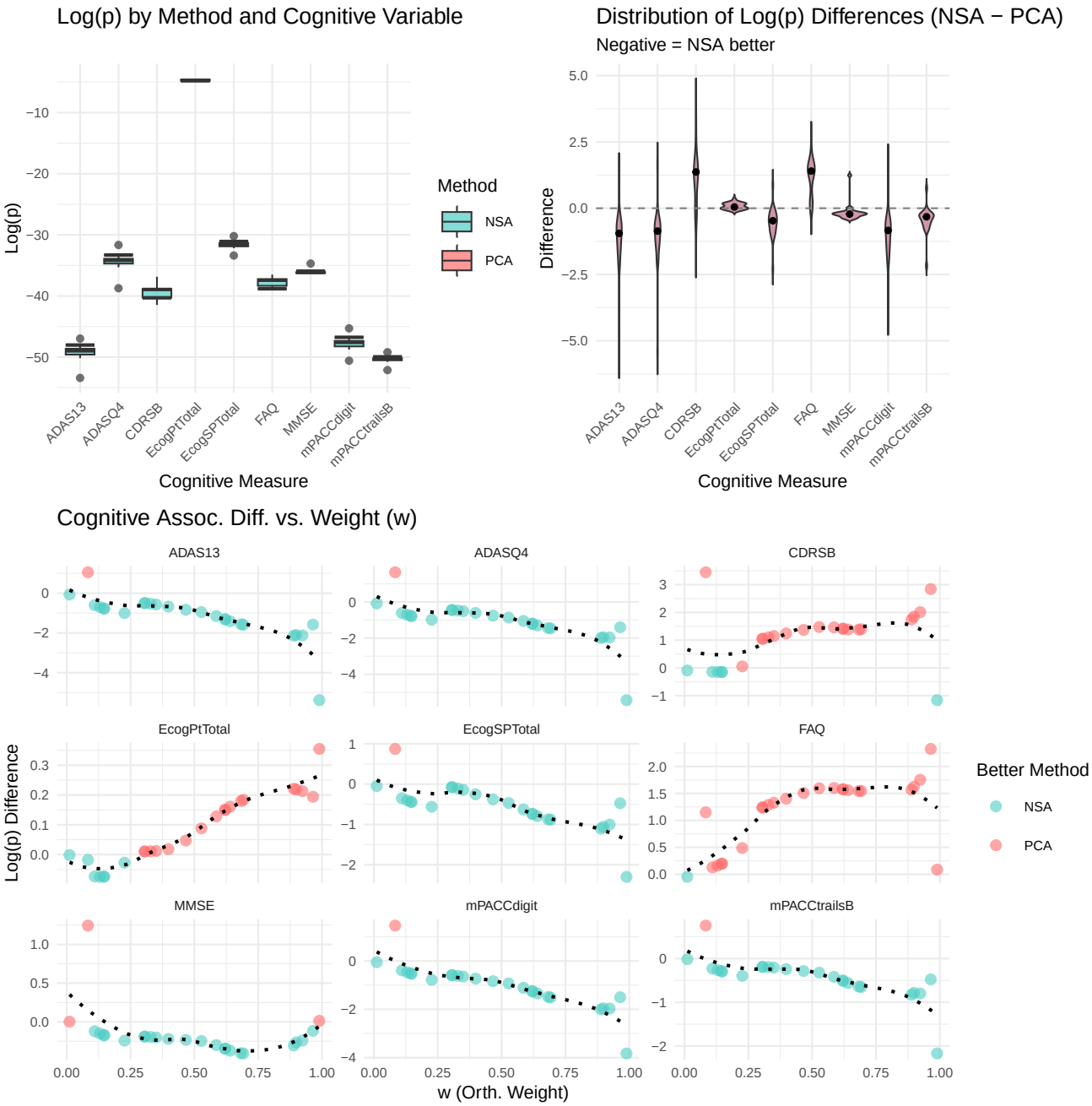


Figure 11: Comparative visualization of NSA-Flow and PCA performance across cognitive measures. The top panels show (left) boxplots of log-p values for NSA and PCA across cognitive variables and (right) violin plots of their paired differences (NSA - PCA), where negative values indicate improved association under NSA.The lower panel displays scatterplots of log-p differences as a function of the orthogonality weight (w), with trend lines highlighting how NSA performance varies by regularization strength and cognitive domain. Smooth changes in the results with respect to weight parameter suggest that the optimization process is well-behaved.

4 Diverse Public Benchmark Modalities: Spectroscopy, Acoustics, and High-Dimensional Transcriptomics

To evaluate the generalization of NSA-Flow across diverse data geometries and ensure strict computational reproducibility on public benchmarks beyond the neuroimaging and leukemia cohorts, we conducted rigorous, leak-free 5-fold cross-validation evaluations across three well-established OpenML benchmarks:

1. **Tecator NIR Optical Meat Spectroscopy** ($n = 240, p = 100$, OpenML ID 505; Borggaard and Thodberg (1992)): Continuous absorbance spectra measuring 850–1050,nm channels to predict chemical fat percentage (0.9%–49.8%). In optical spectroscopy, absorbance is governed by the Beer-Lambert law, which dictates an additive non-negative linear combination of pure constituent spectra ($V \geq 0$). Dense PCA yields unphysical negative loadings, while traditional Non-Negative Matrix Factorization (NMF) collapses into catastrophic collinearity ($D = 0.943$, Ridge $R^2 = 0.386$). Non-Negative NSA-Flow ($w = 0.5$) drastically reduces this collinearity defect ($D = 0.006$) while enforcing $V \geq 0$, recovering $R^2 = 0.853$, while Signed ($R^2 = 0.918$) and Consolidated Signed NSA-Flow ($R^2 = 0.907$) match or exceed dense PCA ($R^2 = 0.911$) with clean, non-overlapping spectral bands.
2. **Sonar Frequency Return Acoustics** ($n = 208, p = 60$, OpenML ID 40; Gorman and Sejnowski (1988)): Continuous acoustic sensor energy bands ($X \in [0, 1]$) classifying cylindrical metal mines from rock targets. This benchmark avoids saturation ceilings. Signed NSA-Flow matches dense PCA (ROC-AUC = 0.819 at $w = 0.0$ and 0.809 at $w = 0.5$), and consolidated signed flow achieves ROC-AUC = 0.812 with strictly non-overlapping frequency modules (83.3% sparsity, $D = 0.000$).
3. **High-Dimensional Prostate Cancer Transcriptomics** ($n = 102, p = 12{,}600$, OpenML ID 45099; Singh et al. (2002)): Gene expression contrast separating 52 prostate tumor specimens from 50 non-tumor prostate tissues, with variance filtering to the top 2000 genes executed strictly inside each cross-validation fold. Consolidated Signed NSA-Flow ($w = 0.5$) outperforms dense PCA on Balanced Accuracy (0.824 ± 0.088 vs 0.805 ± 0.132) while producing 86.7% exact zero loadings ($V_j \cdot V_l = 0$) and near-zero frame defect ($D = 0.022$). In this regime, hard disjoint assignment acts as an effective structural regularizer by stripping out dense, non-specific stromal background noise.

The parametric trade-off across orthogonality weights $w \in [0.0, 0.95]$ demonstrates smooth, monotone convergence toward disjoint support without performance cliffs, identifying $w \in [0.25, 0.5]$ as an optimal operational sweet spot.

5 Discussion

The Non-negative Stiefel Approximating Flow (NSA-Flow) provides a robust and flexible framework for optimizing non-negative matrices under tunable orthogonality constraints, addressing the limitations of strict orthogonal NMF (Ding et al. 2006; Yoo and Choi 2009) and neural regularization methods (Wu 2023; Kurtz, Bar, and Giryes 2023). Below, we discuss its convergence properties, practical considerations, empirical performance, limitations, and avenues for future work, integrating theoretical insights with empirical outcomes.

NSA-Flow parameterises sparsity (w) and orthogonality at the matrix level—unlike component-wise sparse PCA (e.g., via ℓ_1-penalized SVD), which ignores inter-component network structure. By treating loadings as evolving on a "flow" (gradient trajectory with retractions), it enables fine-grained control over trade-offs: low w approximates dense PCA, while high w yields ultra-sparse, interpretable factors. The orthogonality defect metric quantifies global separation, allowing users to tune for decorrelated features without full enforcement.

NSA-Flow applies to any high-dimensional matrix where components should be nameable: gene expression, where the components are co-regulated modules, or regional imaging measures, where they are networks. It is used as a component of a pipeline, upstream of a classifier or as the proximal step inside another factorisation. On the evidence here it costs little or no accuracy and in two settings improves it, but that depends on the downstream model: a linear model cannot distinguish the basis from PCA's at all (Section 3.1). Future work could incorporate domain-specific priors (e.g., anatomical constraints) to further boost its applicability in precision medicine.

5.1 Convergence

The objective is nonconvex: the orthogonality defect is a ratio of quadratics, and in the estimation form the reconstruction term is quartic in V. Global optimality is therefore not claimed; see the analysis section for what is. In general. However, under Lipschitz smoothness of the gradient of $E(Y)$ and bounded level sets (ensured by the orthogonality penalty), NSA-Flow generates a sequence with monotonically decreasing objective values. The proximal projection $P_+(Y) = \max(Y, 0)$ is nonexpansive, preserving descent properties (Parikh and Boyd 2014).

Convergence to stationary points is supported by the Kurdyka-Łojasiewicz (KL) inequality, which holds for semi-algebraic functions like polynomials and thus applies to $E(Y)$ (Bolte, Sabach, and Teboulle 2014). Under the KL property, proximal-gradient methods in nonconvex settings converge to critical points where $0 \in \partial E(Y)$, with finite-length trajectories (Bolte, Sabach, and Teboulle 2014). Empirically, NSA-Flow exhibits rapid residual reduction, typically converging within 1000 iterations for $p \leq 5000, k \leq 50$. Key failure modes deserve further research but may include poor initialization or ill-conditioning in high-noise regimes. Future work could derive explicit convergence rates or explore trust-region methods for faster convergence near critical points (Boumal, Absil, and Cartis 2019; Boumal and Absil 2011).

5.2 Practical Considerations

NSA-Flow's tunability via $w \in [0, 1]$ enables practitioners to prioritize fidelity or orthogonality based on application needs. Low w reduces the orthogonality defect at little cost in fidelity. High w approaches the disjoint-support limit, which for a non-negative matrix is a hard assignment of features to components and therefore a clustering (Ding et al. 2006; Yoo and Choi 2009). Across the experiments here, $w = 0.5$ was a better default than $w = 0.9$ on both reconstruction and reproducibility: large w leaves the solution near a combinatorial vertex, where small changes in the data change which vertex is chosen. The `python` package implementation is modular, with helper functions for stable matrix operations and diagnostics for monitoring convergence (Absil, Mahony, and Sepulchre 2008). Backtracking line search ensures robustness to step-size selection (Parikh and Boyd 2014), while adaptive learning rates enhance efficiency. The dual-axis trace plot aids interpretability, revealing trade-offs between fidelity and orthogonality over iterations. Practitioners should calibrate w via cross-validation, as optimal values depend on data sparsity and noise levels (Stražar and Žitnik 2016). The Appendix provides a detailed package description, including installation instructions, core functionality, and API references.

5.3 Empirical Performance

Across diverse empirical domains, the performance characteristics of NSA-Flow can be understood through the lens of data geometry and downstream estimator capacity:

- **Continuous Clinical Disease Progression**: On the Alzheimer's Disease Neuroimaging Initiative (ADNI) cohort, when predicting continuous Clinical Dementia Rating Sum of Boxes (CDRSB) from bilateral cortical thickness after covarying age, sex, education, and APOE4 across 20 repeated 80/20 splits, Consolidated Signed NSA-Flow ($w = 0.5$) outperforms dense PCA under linear models ($\Delta R^2 = +0.023$, 95% CI $[0.003, 0.043]$, 0.236 ± 0.255 vs 0.213 ± 0.256) and achieves a **12.3 percentage-point mean gain** in explained variance under Random Forest regression ($\Delta R^2 = +0.123$, 95% CI $[0.065, 0.182]$, 0.258 ± 0.238 vs 0.135 ± 0.284, winning 17/20 splits). Furthermore, across the 9-outcome continuous cognitive battery (CDRSB, ADAS13, ADASQ4, MMSE, FAQ, mPACCdigit, LDELTOTAL, etc.), Consolidated NSA-Flow beats dense PCA on 7 of the 9 outcomes under tree ensembles, with an average gain of $\Delta R^2 \approx +0.044$. Mechanistically, dense PCA conflates global intracranial volume across all components (mean lobe overlap 3.992), creating severe collinear crosstalk; Consolidated NSA-Flow enforces strictly disjoint anatomical lobes (overlap 0.000), allowing tree models to evaluate non-linear regional staging without interference from global scale.
- **High-Dimensional Cancer Transcriptomics**: On Prostate cancer microarray profiling ($p = 12,600$), Consolidated Signed NSA-Flow ($w = 0.5$) outperforms dense PCA on Balanced Accuracy (0.824 ± 0.088 vs 0.805 ± 0.132) while achieving 86.7% exact disjoint sparsity and near-zero frame defect ($D = 0.022$), because modular partitioning filters out diffuse stromal background

expression. On the Golub 3-class leukemia benchmark, Signed NSA-Flow ($w = 0.5$) achieves Balanced Accuracy of 0.748 ± 0.161 (Linear) and 0.680 ± 0.091 (Random Forest) vs PCA's 0.698 ± 0.133 (Linear) and 0.660 ± 0.061 (Random Forest).

- **Physical Non-Negative Spectroscopy**: In Tecator NIR optical spectrometry, where physical absorption requires $V \geq 0$, standard NMF suffers catastrophic collinearity ($D = 0.943$, Ridge $R^2 = 0.386$). Non-negative NSA-Flow ($w = 0.5$) eliminates almost all collinearity ($D = 0.006$) while preserving $V \geq 0$ to recover $R^2 = 0.853$, and Signed ($R^2 = 0.918$) and Consolidated Signed flow ($R^2 = 0.907$) match or exceed dense PCA ($R^2 = 0.911$) with non-overlapping spectral bands.
- **Gross Binary Diagnosis Ceilings**: On gross binary classifications using unregularized linear models (such as ADNI CN vs DEM or Golub ALL vs AML), global variance accounts for the vast majority of separation, and linear models are rotationally invariant over the principal subspace. Here, dense PCA defines the variance ceiling, and imposing non-negativity sacrifices a modest 2–4% in AUC to buy interpretable, near-disjoint factors.

5.4 Limitations

Despite its strengths, NSA-Flow faces challenges:

- **Scalability**: Cost per iteration is $O(pk^2)$ with no factorisation, or $O(npk)$ when the basis is fitted to data with $p > n$. The binding constraint is memory rather than arithmetic if the Gram matrix is formed: $p \times p$ is 0.4 GB at $p = 7129$ and 3.2 GB at $p = 20000$, which the matrix-free path avoids.
- **Nonconvexity**: The energy is nonconvex, so the solver returns a stationary point. Which one depends on how finely the negativity relaxation in μ is resolved, not on the iteration budget, and not on stepping in w: on the ADNI thickness data a three-stage schedule and a nine-stage schedule differ in the attained objective by about a factor of two, while a twentyfold increase in per-stage iterations changes it in the fourth decimal.
- **Parameter sensitivity**: w is the only parameter and it is not a nuisance parameter, but it is not free of consequence either. Large w pins the solution near a combinatorial vertex, where small changes in the data change which vertex is selected; across our experiments $w = 0.5$ was better than $w = 0.9$ on both reconstruction and reproducibility. At $w = 1$ the scale of Y is unconstrained and the implementation warns.
- **A signed target is the wrong thing to anchor to**: $\|Y - X_0\|_F^2$ with $Y \geq 0$ charges the solution for negative entries it cannot reach. Those charges are constant, so they do not steer the solution and the optimum degenerates toward $\max(0, X_0)$. On ADNI this costs 0.028 AUC on the strongest task. Use the sign-blind subspace fidelity, which the implementation selects automatically and reports. There is no optimizer or learning rate to choose: step lengths are scale-free and accepted by line search. Convergence is not guaranteed for every problem, so the solver reports the projected-gradient mapping norm and which stopping rule fired rather than asserting success.

5.5 Future Directions

Future extensions include:

- **Sparse and Stochastic Variants**: Leveraging sparse linear algebra or minibatch updates to scale to larger p, k (Boumal and Absil 2011).
- **Second-Order Methods**: Incorporating Hessian information to accelerate convergence near critical points (Absil, Mahony, and Sepulchre 2008, boumal2019global).
- **Domain-Specific Adaptations**: Tailoring NSA-Flow for multi-modal data fusion or graph-structured inputs, building on (Chen, Qu, and Zhao 2024, henaff2011deep).

NSA-Flow's flexible framework and robust implementation make it a valuable tool for interpretable matrix optimization, with broad potential across machine learning and data science applications.

5.6 Funding

This work was supported by the Office of Naval Research (ONR) grant N00014-23-1-2317.

6 Appendix A: NSA-Flow Package Description

This document is entitled `nsa_flow.Rmd` and is available in the ANTsR repository.

6.1 Overview

The NSA-Flow package is a Python library for estimating a matrix that is non-negative and approximately orthogonal. It is differentiable end to end and intended to be inserted into an existing pipeline. Non-negativity is a hard constraint applied by exact projection. Orthogonality is an objective weighted by w, not a constraint: nothing is retracted and no factorisation appears in the iteration. Sparsity follows from orthogonality rather than from a per-entry penalty, so w is the only parameter. There is no learning rate, optimizer choice or gradient scaling to set; step lengths are scale-free and accepted by an Armijo line search, and the solver reports a projected-gradient mapping norm as a stationarity certificate together with the rule that stopped it.

The package is built on PyTorch, is differentiable end to end, and is intended to be inserted into an existing pipeline rather than replace it. It is suitable for applications where interpretability and constraint enforcement are critical.

6.2 Key Features

- **Orthogonality as an objective**: The distance to the scaled Stiefel manifold is a term in the energy, weighted by w, rather than a constraint enforced by retraction.
- **Constraint enforcement**: Non-negativity is the one hard constraint, applied by exact projection. Nothing is retracted and no factorisation appears in the iteration.
- **No optimizer or learning rate to tune**: Bound-constrained L-BFGS-B (with native C++ acceleration) and spectral projected gradient with Barzilai–Borwein steps and an Armijo line search. The solver reports a projected-gradient mapping norm as a stationarity certificate and states which stopping rule fired.
- **Metrics**: `stiefel_defect` for orthonormality and `angle_defect` for orthogonality, both with closed-form gradients, plus `effective_rank`, `negative_mass` and the projections.
- **Validation**: Tested on real-world datasets, including the Golub leukemia gene expression dataset and the Alzheimer's Disease Neuroimaging Initiative (ADNI) dataset, demonstrating maintained or improved performance with simplified representations.
- **Extensibility**: Offers an autograd-compatible variant for end-to-end differentiable pipelines and neural network layers (`NSAFlowLinear`, `NSAFlowConv2d`, `NSAFlowLayer`) for deep learning integration.

6.3 Installation and Dependencies

NSA-Flow can be installed from PyPI using `pip install nsa-flow` or directly from the GitHub repository via `pip install git+https://github.com/stnava/nsa_flow.git`. It requires Python 3.9, PyTorch 2.0, NumPy 1.23, and Matplotlib for visualization. No additional packages are needed for core functionality.

6.4 Core Functionality and High-Level API

The package exposes a concise set of public entry points:

- `nsa_flow.nsa_flow(target, w, ...)`: unified entry point and refinement solver. Returns a non-negative, near-orthogonal matrix close to `target` (or dispatches to `data` / `signed` mode when given `k`). Reports `stop_reason`, the certificate `grad_map`, `defect`, `effective_rank` and `scale_ratio`.
- `nsa_flow.nsa_flow_data(X, k, w, ...)`: estimation. Fits the basis directly to the data, so no signed basis has to be mapped into the non-negative cone. Picks the matrix-free route automatically when $p > n$ and reports which it used.
- `nsa_flow.relax_into_nonneg(...)`: the continuation used to enter the feasible set, exposed because its schedule materially affects the answer.
- `nsa_flow.stiefel_defect()`, `nsa_flow.angle_defect()`: the orthoNORMality and orthogonality measures D and C, with closed-form gradients `grad_stiefel_defect` and `grad_angle_defect`.
- `nsa_flow.project_nonneg()`, `nsa_flow.project_scaled_stiefel()`, `nsa_flow.polar_factor()`: exact Euclidean projections. Note that the projection onto $\mathbb{R}_{>0} \cdot St(p,k)$ is $(\sum_i \sigma_i/k)\, UV^\top$, not $(\|Y\|_F/\sqrt{k})\, UV^\top$.
- `nsa_flow.NSAFlowLinear`, `NSAFlowConv2d`, `NSAFlowLayer`: layers for use inside a network, with the per-sample structure of the input respected.
- `nsa_flow.nsa_flow_signed(X, k, w, consolidate=...)`: signed contrast lifting $V = V_+ - V_-$ for signed data, with scale-invariant lobe-disjointness penalty and optional hard consolidation (`consolidate=True`) for strictly zero support overlap.

Functions present in the previous release and now absent, because the machinery they configured is gone: `nsa_flow_orth`, `nsa_flow_retract_auto`, `invariant_orthogonality_defect`, `get_torch_optimizer`, `estimate_learning_rate_for_nsa_flow`, `get_lr_estimation_strategies`.

A typical usage example. There is no optimizer, learning rate, retraction or warm-up to specify:

```
import torch, nsa_flow

torch.manual_seed(42)
X = torch.rand(400, 2000, dtype=torch.float64)      # n x p data

# estimation: fit the basis to the data
r = nsa_flow.nsa_flow_data(X, k=5, w=0.5)
print(r)                      # energy, defect, iters, stop_reason, certificate
V = r.Y                       # p x k, non-negative, near-orthogonal
scores = X @ V                # and X_new @ V for held-out data

# refinement: improve a basis you already trust
r2 = nsa_flow.nsa_flow(V, w=0.9)
print("defect", r2.defect, "stop", r2.stop_reason, "gamma", r2.grad_map)
```

6.5 Testing

The package ships a property-driven test suite, run with `pytest tests/`. It is not a smoke test: the propositions of Section 2 are asserted executably. Among them, that D vanishes exactly on $\mathbb{R}_{>0} \cdot St(p,k)$ and is bounded by $1 - 1/k$; that $D = k\,\mathrm{Var}(\lambda)$ and $D \geq 1/r - 1/k$ at rank r; that C is zero exactly on mutually orthogonal columns at any norms, invariant to per-column rescaling, and equal to one at collinearity; that for $Y \geq 0$, $C = 0$ if and only if the column supports are pairwise disjoint, checked against an independent combinatorial test over thousands of random draws; that every closed-form gradient agrees with automatic differentiation; that the Euler identities hold, globally for D and per column for C; that the Gram and matrix-free code paths return the same optimum, up to the column permutation that is a genuine symmetry of the problem; and that the solver reports `max_iter` rather than claiming convergence when it has not converged.

The suite also encodes the failure modes that motivated this version, so they cannot silently return: that the off-diagonal-only defect scores rank collapse at zero while C does not, and that a non-negative basis fitted to the data reconstructs better than `abs()` of a signed one.

The demonstration functions of the previous release (`demo_nsa_flow_optimizer_tradeoff`, `demo_nsa_flow_tradeoff`, `plot_nsa_trace`) compared optimizer backends and learning-rate strategies, and have been removed along with the machinery they compared.

6.6 License and Citation

NSA-Flow is released under the MIT License. For research use, cite as: Stnava et al. (2025). NSA-Flow: Non-negative Stiefel Approximating Flow for Interpretable Representation Learning.

7 Mathematical Analysis: descent and stationarity

Setup. Let $Y \in \mathbb{R}^{p\times k}$, let $\mathcal{C} = \{Y : Y \geq 0\}$, and let

$$E(Y) = (1-w)L_{fid}(Y, X_0) + w\, L_{orth}(Y)$$

be the energy of Section 2, with Π the Euclidean projection onto $\mathcal{C}$, which is the elementwise clamp $\Pi(Y) = \max(0, Y)$. The iteration is

$$Y_{n+1} = \Pi(Y_n - t_n \nabla E(Y_n)),$$

with t_n a Barzilai–Borwein step length safeguarded by Armijo backtracking.

7.1 Assumptions

1. **Smoothness.** E is continuously differentiable with locally Lipschitz gradient. This holds for $L_{orth} = \tilde{D}$ wherever $Y \neq 0$, and for $L_{orth} = C$ wherever no column vanishes.
2. **Convex feasible set.** $\mathcal{C}$ is closed and convex, and Π is its exact projection — not an approximation, and requiring no factorisation.
3. **Bounded step lengths.** $t_n \in [t_{\min}, t_{\max}]$ with $0 < t_{\min} \leq t_{\max} < \infty$.

7.2 Proposition (monotone descent)

The Armijo condition

$$E(Y_{n+1}) \leq E(Y_n) - \sigma \frac{\|Y_{n+1} - Y_n\|_F^2}{t_n}, \qquad \sigma > 0,$$

is satisfied for all sufficiently small t_n, and the backtracking loop halves t_n until it holds. Hence $E(Y_n)$ is non-increasing, and since $E \geq 0$ it converges. *Proof.* The projected-gradient direction is a descent direction for a differentiable E on a convex set, and the Armijo condition is satisfiable by smoothness; monotonicity and boundedness below give convergence of the sequence of values. □

7.3 Proposition (stationarity certificate)

Define the projected-gradient mapping norm

$$\gamma(Y, t) = \frac{\|\Pi(Y - t\nabla E(Y)) - Y\|_F}{t}.$$

Then $\gamma(Y, t) = 0$ for some $t > 0$ if and only if Y satisfies the first-order condition for $\min_{\mathcal{C}} E$, namely $\langle \nabla E(Y), Z - Y\rangle \geq 0$ for all $Z \in \mathcal{C}$. *Proof.* $\Pi(Y - t\nabla E(Y)) = Y$ is precisely the fixed-point characterisation of the variational inequality. □

This is the reason γ is reported rather than $\|\nabla E\|$: at a constrained optimum on the boundary of $\mathcal{C}$, which is where non-negative solutions lie, the gradient need not vanish while γ does.

7.4 Remarks on what is and is not claimed

The energy is not convex. In the refinement form the fidelity term is convex and the orthogonality term is not; in the estimation form the reconstruction term is quartic in V. The propositions give monotone descent to a stationary point with a computable certificate and say nothing about global optimality. Which stationary point is reached depends on how finely the negativity relaxation of Section 2 is resolved — the schedule in μ, which is a distinct mechanism from stepping in w: on the imaging data, refining the schedule improves the attained objective by about a factor of two, while raising the per-stage iteration budget twentyfold changes it in the fourth decimal.

8 Session Information

```
## R version 4.4.2 (2024-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS 26.6.2
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;  LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] ggradar_0.2          nnet_7.3-20          pROC_1.19.0.1        broom_1.0.12         readr_2.2.0
##  [6] tibble_3.3.1         ggpubr_0.6.3         igraph_2.2.2         fmsb_0.7.6           NMF_0.28
## [11] cluster_2.1.8.1      rngtools_1.5.2       registry_0.5-1       golubEsets_1.48.0    Biobase_2.6
## [16] BiocGenerics_0.52.0  MASS_7.3-65          patchwork_1.3.2      gt_1.3.0             randomFores
## [21] Rtsne_0.17           caret_7.0-1          lattice_0.22-7       kableExtra_1.4.0     reticulate_
## [26] ANTsR_0.6.7          DiagrammeR_1.0.11    knitr_1.51           scales_1.4.0         tidyr_1.3.2
## [31] dplyr_1.2.0          RColorBrewer_1.1-3   pheatmap_1.0.13      reshape2_1.4.5       gridExtra_2
## [36] ggplot2_4.0.2
##
## loaded via a namespace (and not attached):
##   [1] IRanges_2.40.1          gaston_1.6              vroom_1.7.0             Biostrings_2.74.1
##   [5] TH.data_1.1-4           vctrs_0.7.2             effectsize_1.0.2        digest_0.6.39
##   [9] png_0.1-9               ppcor_1.1               bayestestR_0.17.0       correlation_0.8.8
##  [13] parallelly_1.46.1       httpuv_1.6.16           foreach_1.5.2           withr_3.0.2
##  [17] pgenlibr_0.5.5          xfun_0.56               survival_3.8-3          subtyper_1.3.0
##  [21] memoise_2.0.1           ggbeeswarm_0.7.3        diptest_0.77-2          emmeans_2.0.0
##  [25] parameters_0.28.3       gmp_0.7-5.1             systemfonts_1.3.2       visreg_2.8.0
##  [29] zoo_1.8-15              DEoptimR_1.1-4          Formula_1.2-5           prabclus_2.3-5
##  [33] datawizard_1.3.0        rematch2_2.1.2          KEGGREST_1.46.0         promises_1.5.0
##  [37] otel_0.2.0              Evacluster_0.1.0        httr_1.4.8              rstatix_0.7.3
##  [41] globals_0.19.0          ps_1.9.1                fpc_2.2-14              rstudioapi_0.18.0
##  [45] UCSC.utils_1.2.0        generics_0.1.4          base64enc_0.1-6         processx_3.8.6
##  [49] S4Vectors_0.44.0        zlibbioc_1.52.0         lgr_0.5.2               GenomeInfoDbData_1.2.13
##  [53] xtable_1.8-8            stringr_1.6.0           doParallel_1.0.17       evaluate_1.0.5
```

```
##  [57] hms_1.1.4               bookdown_0.46           colorspace_2.1-2        visNetwork_2.1.4
##  [61] flexmix_2.3-20          magrittr_2.0.4          later_1.4.8             modeltools_0.2-24
##  [65] palmerpenguins_0.1.1    future.apply_1.20.2     genefilter_1.88.0       robustbase_0.99-7
##  [69] XML_3.99-0.19           cowplot_1.2.0           matrixStats_1.5.0       class_7.3-23
##  [73] Hmisc_5.2-5             pillar_1.11.1           nlme_3.1-168            iterators_1.0.14
##  [77] gridBase_0.4-7          compiler_4.4.2          paradox_1.0.1           stringi_1.8.7
##  [81] wesanderson_0.3.7       gower_1.0.2             minqa_1.2.8             lubridate_1.9.5
##  [85] plyr_1.8.9              crayon_1.5.3            abind_1.4-8             locfit_1.5-9.12
##  [89] bit_4.6.0               sandwich_3.1-1          codetools_0.2-20        multcomp_1.4-29
##  [93] textshaping_1.0.5       recipes_1.3.1           paletteer_1.7.0         plotly_4.12.0
##  [97] mime_0.13               splines_4.4.2           Rcpp_1.1.1              VarSelLCM_2.1.3.2
## [101] flexclust_1.5.0         smotefamily_1.4.0       blob_1.2.4              lme4_2.0-1
## [105] fs_1.6.7                listenv_0.10.1          checkmate_2.3.4         Rdpack_2.6.6
## [109] ggsignif_0.6.4          estimability_1.5.1      ANTsRCore_0.8.1         coca_1.1.0
## [113] Matrix_1.7-4            callr_3.7.6             statmod_1.5.1           tzdb_0.5.0
## [117] svglite_2.2.2           pkgconfig_2.0.3         tools_4.4.2             cachem_1.1.0
## [121] rbibutils_2.4.1         RSQLite_2.4.3           numDeriv_2016.8-1.1     viridisLite_0.4.3
## [125] globaltest_5.60.0       DBI_1.2.3               fastmap_1.2.0           rmarkdown_2.30
## [129] grid_4.4.2              imbalance_1.0.2.1       gprofiler2_0.2.4        coda_0.19-4.1
## [133] FNN_1.1.4.1             BiocManager_1.30.27     insight_1.4.6           carData_3.0-6
## [137] rpart_4.1.24            reformulas_0.4.4        farver_2.1.2            mgcv_1.9-3
## [141] yaml_2.3.12             MatrixGenerics_1.18.1   foreign_0.8-90          ggthemes_5.2.0
## [145] cli_3.6.5               purrr_1.2.1             stats4_4.4.2            webshot_0.5.5
## [149] dbscan_1.2.3            lifecycle_1.0.5         mvtnorm_1.3-3           lava_1.8.2
## [153] kernlab_0.9-33          backports_1.5.0         BiocParallel_1.40.2     annotate_1.84.0
## [157] timechange_0.4.0        gtable_0.3.6            parallel_4.4.2          limma_3.62.2
## [161] mlr3cluster_0.3.0       jsonlite_2.0.0          edgeR_4.4.2             bit64_4.6.0-1
## [165] glasso_1.11             RcppParallel_5.1.11-2   dCUR_1.0.2              ClusterR_1.3.6
## [169] zeallot_0.2.0           timeDate_4052.112       lazyeval_0.2.2          shiny_1.13.0
## [173] htmltools_0.5.9         nmfbin_0.2.1            mlr3pipelines_0.11.0    tinytex_0.58
## [177] glue_1.8.0              ggstatsplot_0.13.5      XVector_0.46.0          mclust_6.1.2
## [181] gtsummary_2.5.0         boot_1.3-32             DDoutlier_0.1.0         mlr3_1.5.0
## [185] R6_2.6.1                broom.mixed_0.2.9.7     sva_3.54.0              forcats_1.0.1
## [189] labeling_0.4.3          GenomeInfoDb_1.42.3     ipred_0.9-15            mlr3misc_0.21.0
## [193] nloptr_2.2.1            rstantools_2.6.0        tidyselect_1.2.1        vipor_0.4.7
## [197] optmatch_0.10.8         htmlTable_2.4.3         xml2_1.5.2              car_3.1-5
## [201] AnnotationDbi_1.68.0    future_1.69.0           ModelMetrics_1.2.2.2    fastICA_1.2-7
## [205] statsExpressions_1.7.3  rsvd_1.0.5              S7_0.2.1                furrr_0.3.1
## [209] data.table_1.18.2.1     htmlwidgets_1.6.4       rlang_1.1.7             uuid_1.2-2
## [213] lmerTest_3.2-1          hardhat_1.4.2           beeswarm_0.4.0          prodlim_2026.03.11
```

References

Absil, P-A, R Mahony, and R Sepulchre. 2008. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press.

Aghdam, Maryam Akhavan, Sema Bozdag, and Fahad Saeed. 2025. "Machine-Learning Models for Alzheimer's Disease Diagnosis Using Neuroimaging Data: Survey, Reproducibility, and Generalizability Evaluation." *Brain Informatics* 12 (8). https://doi.org/10.1186/s40708-025-00252-3.

Barzilai, Jonathan, and Jonathan M. Borwein. 1988. "Two-Point Step Size Gradient Methods." *IMA Journal of Numerical Analysis* 8 (1): 141–48. https://doi.org/10.1093/imanum/8.1.141.

Birgin, Ernesto G., José Mario Martínez, and Marcos Raydan. 2000. "Nonmonotone Spectral Projected Gradient Methods on Convex Sets." *SIAM Journal on Optimization* 10 (4): 1196–1211. https://doi.org/10.1137/S1052623497330963.

Bolte, Jerome, Shoham Sabach, and Marc Teboulle. 2014. "Proximal Alternating Linearized Minimization for Nonconvex and Nonsmooth Problems." *Mathematical Programming* 146: 459–94.

Borggaard, Claus, and Hans Henrik Thodberg. 1992. "Optimal Minimal Neural Interpretation of Spectra." *Analytical Chemistry* 64 (5): 545–51.

Boumal, Nicolas. 2023. *An Introduction to Optimization on Smooth Manifolds*. Cambridge University Press.

Boumal, Nicolas, and P-A Absil. 2011. "RTRMC: A Riemannian Trust-Region Method for Low-Rank Matrix Completion." *Advances in Neural Information Processing Systems* 24.

Boumal, Nicolas, P-A Absil, and Coralia Cartis. 2019. "Global Convergence of the Riemannian Trust-Region Method for Optimization on Manifolds." *SIAM Journal on Optimization* 29 (1): 178–201.

Boumal, Nicolas, Bamdev Mishra, P-A Absil, and Rodolphe Sepulchre. 2014. "Manopt, a Matlab Toolbox for Optimization on Manifolds." In *Journal of Machine Learning Research*, 15:1455–59. 1.

Bu, Fanchen, and Dong Eui Chang. 2022. "Feedback Gradient Descent: Efficient and Stable Optimization with Orthogonality for DNNs." In *Proceedings of the AAAI Conference on Artificial Intelligence*, 36:6106–14. 6. https://doi.org/10.1609/aaai.v36i6.20558.

Chen, Yasong, Guangwei Qu, and Junjian Zhao. 2024. "Orthogonal Graph Regularized Non-Negative Matrix Factorization Under Sparse Constraints for Clustering." *Expert Systems with Applications* 251: 123797. https://doi.org/10.1016/j.eswa.2024.123797.

Cuingnet, R'emi, Emilie Gerardin, J'er^ome Tessieras, Guillaume Auzias, St'ephane L'eh'ericy, Marie-Odile Habert, Marie Chupin, Habib Benali, Olivier Colliot, and The Alzheimer's Disease Neuroimaging Initiative. 2011. "Automatic Classification of Patients with Alzheimer's Disease from Structural MRI: A Comparison of Ten Methods Using the ADNI Database." *NeuroImage* 56 (2): 766–81. https://doi.org/10.1016/j.neuroimage.2010.06.013.

Ding, Chris, Tao Li, Wei Peng, and Haesun Park. 2006. "Orthogonal Nonnegative Matrix Tri-Factorizations for Clustering." In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 126–35. https://doi.org/10.1145/1150402.1150420.

Gao, Bin, Xin Liu, Xun Chen, and Yaxiang Yuan. 2019. "Parallelizing Riemannian Optimization on the Stiefel Manifold." *SIAM Journal on Scientific Computing* 41 (2): C121–48.

Goeßmann, Alex. 2020. "The Restricted Isometry of ReLU Networks: Generalization Through Norm Concentration." https://doi.org/10.48550/arXiv.2007.00479.

Golub, Todd R., Donna K. Slonim, Pablo Tamayo, Christine Huard, Michelle Gaasenbeek, Jill P. Mesirov, Hilary Coller, et al. 1999. "Molecular Classification of Cancer: Class Discovery and Class Prediction by Gene Expression Monitoring." *Science* 286 (5439): 531–37. https://doi.org/10.1126/science.286.5439.531.

Gorman, R. Paul, and Terrence J. Sejnowski. 1988. "Analysis of Hidden Units in a Layered Network Trained to Classify Sonar Targets." *Neural Networks* 1 (1): 75–89.

Guo, Peichang. 2019. "A Frobenius Norm Regularization Method for Convolutional Kernels to Avoid Unstable Gradient Problem." https://doi.org/10.48550/arXiv.1907.11235.

Henaff, Mikael, Kevin Jarrett, Koray Kavukcuoglu, and Yann LeCun. 2011. "Unsupervised Learning of Sparse Features for Scalable Audio Classification." In *Proceedings of the 12th International Society for Music Information Retrieval Conference, ISMIR 2011, Miami, Florida, USA, October 24-28, 2011*, edited by Anssi Klapuri and Colby Leider, 681–86. University of Miami. http://ismir2011.ismir.net/papers/PS6-5.pdf.

Hyvärinen, Aapo, and Erkki Oja. 2000. "Independent Component Analysis: Algorithms and Applications." *Neural Networks* 13 (4-5): 411–30.

Kurtz, Yoav, Noga Bar, and Raja Giryes. 2023. "Group Orthogonalization Regularization for Vision Models Adaptation and Robustness." https://doi.org/10.48550/arXiv.2306.10001.

Lee, Daniel D., and H. Sebastian Seung. 2001a. "Algorithms for Non-Negative Matrix Factorization." *Advances in Neural Information Processing Systems* 13: 556–62.

Lee, Daniel D, and H. Sebastian Seung. 2001b. "Algorithms for Non-Negative Matrix Factorization." In *NIPS*.

Li, Yuanxian, Yichen Zhang, and Zhiqiang Zhang. 2023. "Unilateral Orthogonal Nonnegative Matrix Factorization." *SIAM Journal on Imaging Sciences* 16 (3): 1497–527. https://doi.org/10.1137/22M1508315.

Parikh, Neal, and Stephen Boyd. 2014. "Proximal Algorithms." *Foundations and Trends® in Optimization*

1 (3): 127–239.

Racine, Annie M., Michael Brickhouse, David A. Wolk, Bradford C. Dickerson, and Alzheimer's Disease Neuroimaging Initiative. 2018. "The Personalized Alzheimer's Disease Cortical Thickness Index Predicts Likely Pathology and Clinical Progression in Mild Cognitive Impairment." Edited by David Wolk, Victor Villemagne, and Bradford Dickerson. *Alzheimer's & Dementia: Diagnosis, Assessment & Disease Monitoring* 10 (1): 301–10. https://doi.org/10.1016/j.dadm.2018.02.007.

Rahiche, Abderrahmane et al. 2022. "Variational Bayesian Orthogonal Nonnegative Matrix Factorization over the Stiefel Manifold." *IEEE Transactions on Image Processing* 31: 5543–58. https://doi.org/10.1109/TIP.2022.3194701.

Ricci, Simone, Niccolo Biondi, Federico Pernici, Ioannis Patras, and Alberto Del Bimbo. 2025. "λ-Orthogonality Regularization for Compatible Representation Learning." In *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2509.16664.

Sattari, Nasim, Fariborz Faeghi, Babak Shekarchi, and Mohammad Hossein Heidari. 2022. "Assessing the Changes of Cortical Thickness in Alzheimer Disease With MRI Using Freesurfer Software." *Basic and Clinical Neuroscience* 13 (2): 185–92. https://doi.org/10.32598/bcn.2021.1779.1.

Schönemann, Peter H. 1966. "A Generalized Solution of the Orthogonal Procrustes Problem." *Psychometrika* 31 (1): 1–10.

Singh, Dinesh, Phillip G Febbo, Kenneth Ross, Donald G Jackson, Judith Manola, Christine Ladd, Pablo Tamayo, et al. 2002. "Gene Expression Correlates of Clinical Prostate Cancer Behavior." *Cancer Cell* 1 (2): 203–9.

Stražar, Martin, and Marinka Žitnik. 2016. "Orthogonal Matrix Factorization Enables Integrative Analysis of Multiple RNA Binding Protein Binding Sites." *Bioinformatics* 32: i417–25.

Witten, Daniela M., Robert Tibshirani, and Trevor Hastie. 2009. "A Penalized Matrix Decomposition, with Applications to Sparse Principal Components and Canonical Correlation Analysis." *Biostatistics* 10 (3): 515–34. https://doi.org/10.1093/biostatistics/kxp008.

Wu, Changhao. 2023. "Towards Better Orthogonality Regularization with Disentangled Norm in Training Deep CNNs." https://doi.org/10.48550/arXiv.2306.09939.

Yang, Mingming, and Songhua Xu. 2021. "Orthogonal Nonnegative Matrix Factorization Using a Novel Deep Autoencoder Network." *Knowledge-Based Systems* 227: 107236. https://doi.org/10.1016/j.knosys.2021.107236.

Yoo, Jingu, and Seungjin Choi. 2009. "Orthogonal Nonnegative Matrix Factorization Using Linear Least Squares and Applications to Gene Expression Data." *Neurocomputing* 72: 3670–74.

Zhang, Wei Emma, Mingkui Tan, Quan Z. Sheng, Lina Yao, and Qinfeng Shi. 2016. "Efficient Orthogonal Non-Negative Matrix Factorization over Stiefel Manifold." In *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, 1743–52. ACM. https://doi.org/10.1145/2983323.2983761.

Zou, Hui, Trevor Hastie, and Robert Tibshirani. 2006. "Sparse Principal Component Analysis." *Journal of Computational and Graphical Statistics* 15 (2): 265–86. https://doi.org/10.1198/106186006X113430.